

온라인 플랫폼 관리자 위한 불만 우선순위 자동 분류 및 대응 시스템

Kuggle_2차 프로젝트_분류

13기 손유성 안시현 이현정
14기 박안석 박세준 이지훈



Contents

- 01 프로젝트 개요 및 과정 설명
- 02 데이터 수집
- 03 데이터 병합 및 전처리
- 04 특징 추출 및 사전 구축
- 05 분류 모델 구축
- 06 우선순위 점수 산출
- 07 비즈니스 인사이트 및 전략 추출



분석 목적



[온라인 플랫폼 관리자를 위한 불만 우선순위 자동 분류 및 대응 시스템]

매일 누적되는 리뷰 데이터 속에서 관리자가 주목해야 할 핵심 리뷰를 선정하고,
이에 기반한 후속 액션까지 연결하기 위해 자동화된 분류모델을 활용한
의사결정 지원 파이프라인을 구축해보았습니다.

분석 과정



크롤링 대상 플랫폼을 선정하여 RAW 데이터를 수집



이상치 처리 및 데이터 불균형 해소 방안을 포함한 전처리 수행



리뷰 텍스트 분석을 위한 사전어를 구축하고, 이를 기반으로 긍,부정 분류에 활용할 피처 생성



다양한 분류 모델을 후보로 설정하고, 평가지표를 기반으로 성능을 비교하며
하이퍼파라미터 튜닝을 통해 모델을 고도화



최종 선정된 모델의 테스트 결과를 바탕으로 관리자의 액션 플랜 수립

제품 선택 과정



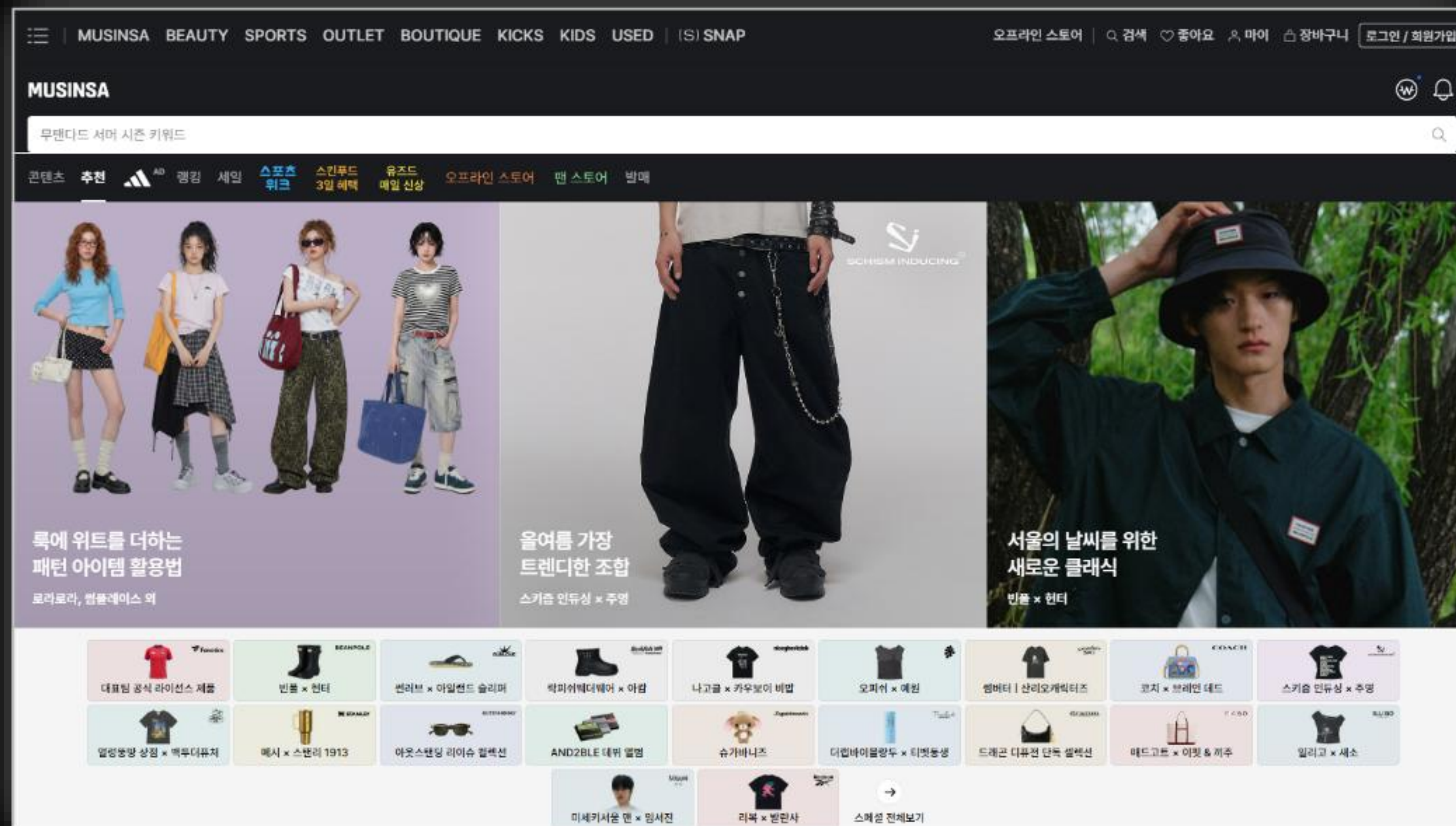
Q. 왜 무신사를 골랐는지?

1. 고도화 된 리뷰 시스템과 커뮤니티성

단순 별점 평가 X
작성자의 키, 몸무게, 구매 사이즈 및 제품의 색감·핏·두께감을 다각도로 평가하는 시스템이 구축되어 있음.
따라서 텍스트와 정량적 체형 정보를 결합한 고차원 분석을 진행하기에 가장 최적화된 플랫폼.

2. 입점 브랜드의 '즉각성 피드백 시스템' 필요성

무신사는 수많은 중소·신진 디자이너 브랜드가 입점해 있음. 대기업과 달리 이들은 CS(고객 서비스/고객 만족)나 QA(품질보증) 인력이 부족하므로, 플랫폼 차원에서 부정 리뷰의 원인을 실시간으로 짚어주는 우선순위 인사이트가 비즈니스적으로 가장 시급하게 요구되는 곳.



제품 선택 과정



Q. 왜 '수아레'라는 브랜드를 골랐는지

1. 복합적 불만이 발생하기 쉬운 '니트(Knit) 품종의 특수성

니트 웨어는 의류 카테고리 중에서도 특히 실 불량(보풀/원사 교체), 세탁 후 수축 및 도안 실측 오류(핏/체형 불만), 실물 색감 차이(정보 비대칭) 등 공정과 기획 전반에 걸친 복합적인 문제가 가장 자주 터지는 품목.

2. 체형 민감도가 높은 아이템

니트는 체형(과체중/저체중 등)에 따라 핏의 만족도가 극명하게 갈리는 제품군임. 표준 체형 위주의 대량 생산 방식이 가진 한계점과 타겟 세분화 문제를 증명하고, 이를 해결하기 위한 '우선순위 가이드라인' 모델을 실험하기에 가장 완벽한 텍스트북(Textbook) 케이스라 판단.



크롤링 제품 선정 과정



①

S U A R E

홈 콘텐츠 발매 전체 상의 바지 아우터 신발 소품 아울렛 기획전

전체 반소매 티셔츠 셔츠/블라우스 민소매 티셔츠 긴소매 티셔츠 니트/스웨터 맨투맨/스웨트

스페셜 무배당발 M 단독

남 여 컬러 패턴/무늬 소재 스타일 가격 사이즈 핏 혜택 상세옵션 상품유형

116개 후기순

④

데일리 라운드 니트 - 12 COLOR
34% 29,900원
♥18만 ★4.9(4만+)
단독

하프 터틀넥 니트 세트
50% 24,900원
♥6.1만 ★4.8(3만+)
단독 아울렛

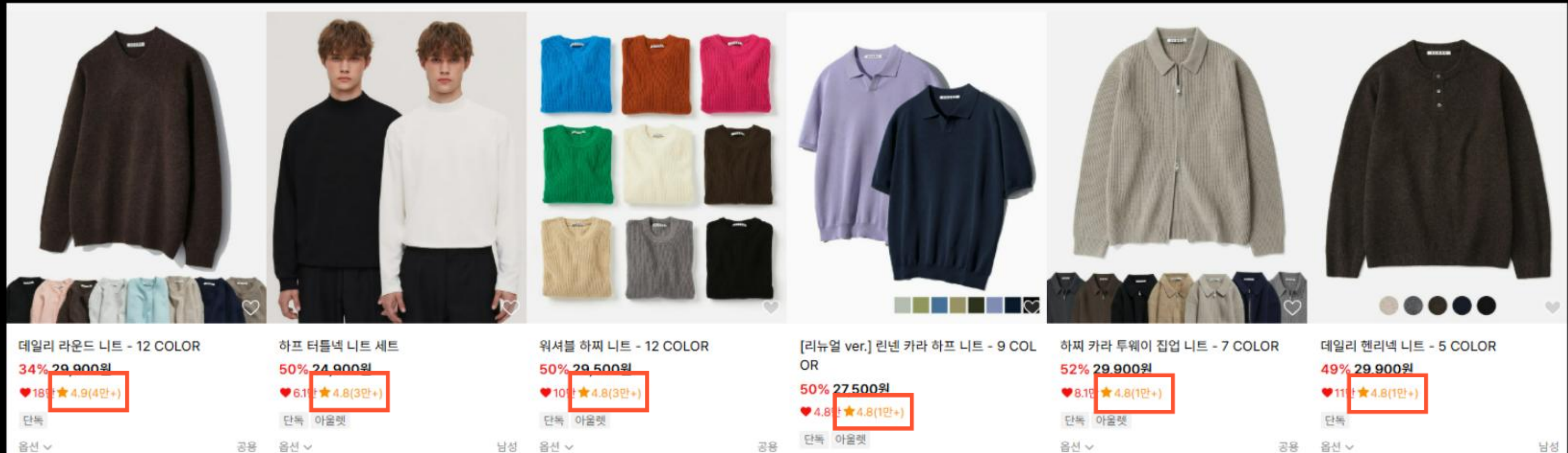
워셔블 하피 니트 - 12 COLOR
50% 29,500원
♥10만 ★4.8(3만+)
단독 아울렛

[리뉴얼 ver.] 린넨 카라 하프 니트 - 9 COLOR
50% 27,500원
♥4.8만 ★4.8(1만+)
단독 아울렛

하피 카라 투웨이 집업 니트 - 7 COLOR
52% 29,900원
♥8.1만 ★4.8(1만+)
단독 아울렛

데일리 헨리넥 니트 - 5 COLOR
49% 29,900원
♥11만 ★4.8(1만+)
단독

크롤링 제품 선정 과정



쇼핑몰 브랜드의 리뷰 특성 상 긍정 리뷰와 부정 리뷰의 개수가 매우 불균형적



- 코드를 2개로 짜서 우선적으로 low rating 하여 1점-2점 리뷰 데이터를 수집한 후, high rating 하여 low rating 리뷰 개수의 약 4배 정도를 크롤링. 즉, low : high = 1 : 4 비율
- 이때 긍정리뷰는 별점 기준으로 4,5점 / 부정 리뷰는 1,2점으로 하였음. 3점은 해석에 모호하므로 제외.

크롤링한 피쳐 설명



25.12.23

★★★★★ 5

상품명 데일리 라운드 니트 - 12 COLOR

구매옵션 BROWN · M

체형정보 남성 · 167cm · 65kg

만족도 사이즈 정사이즈 · 색감 화면과 비슷 · 신축성 강함 · 퀄리티 좋음



정핏으로 이쁘게 떨어지고

원단도 째짤해서 편하게 입기 좋아요

👍 도움돼요 75

💬 댓글 11

1. 구매자 기반 정보 (고객 프로필)

- user_gender: 구매자의 성별
- user_height: 구매자의 키
- user_weight: 구매자의 몸무게
- purchase_size: 구매자가 선택한 의류 사이즈

2. 상품 평가 정보 (정량적 평표)

- rating : 별점
- satisfaction: 무신사 특유의 만족도 평가 항목 (사이즈 / 색감 / 품질)

3. 리뷰 본문 및 피드백 정보 (정성적 데이터 및 파급력)

- review_text : 소비자가 작성한 텍스트 본문
- review_date: 리뷰 작성 날짜
- helpful_count: 해당 리뷰에 공감한 다른 유저들의 수
- is_photo: 사진 첨부 여부

크롤링 코드 분석



musinsa_high	2026-05-24 오후 11:28	Python 원본 파일	8KB
musinsa_low	2026-05-25 오전 12:51	Python 원본 파일	8KB

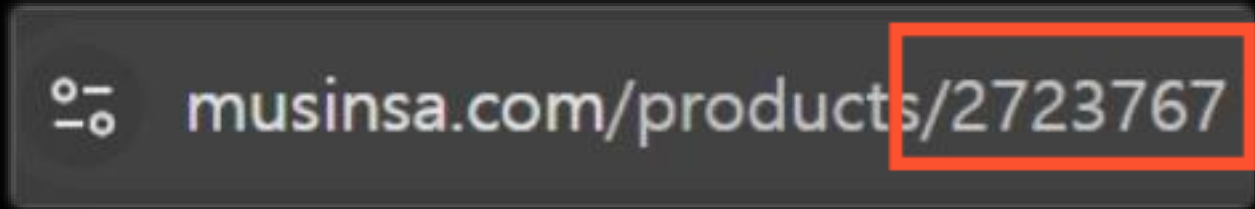
1.브라우저 엔진 설정

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
import time
import csv
import os
import re

# 1. 브라우저 엔진 설정: 자동화 차단 회피 및 성능 최적화
options = Options()
options.add_argument("--headless")
options.add_argument("--window-size=1920,1080").
options.add_argument("--disable-blink-features=AutomationControlled")
options.add_experimental_option("excludeSwitches", ["enable-automation"])
options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36")

driver = webdriver.Chrome(options=options)
```

2. 제품별 pid 할당



제품별로 유니크하게 할당된 제품번호가 존재 (2723767)

```
# 2. 파일 저장 경로 및 대상 상품 식별자 설정
p_id = "2723763"
save_path = r"C:\Users\son76\Documents\2026_4학년1학기\kuggle_2026"
file_name = os.path.join(save_path, f"musinsa_low_{p_id}_final.csv")
```

3. 정렬 기준 설정

```
try:
    # 3. 낮은 점수 순 정렬 파라미터 적용 접속 (sort=goods_est_asc)
    # 무신사 정책상 '별점 없음'은 내부적으로 5점으로 처리되나 낮은 순 정렬 시 가장 먼저 노출됨
    driver.get(f"https://www.musinsa.com/review/goods/{p_id}?sort=goods_est_asc")
    time.sleep(5) # 동적 콘텐츠 렌더링을 위한 충분한 초기 대기

    scraped_count = 0
    scraped_ids = set()

    print(f"[{p_id}] 최종 데이터 수집 시작 (대상: 별점 없음, 1점, 2점)...")
```


4. 데이터 수집

```

for box in boxes:
    try:
        review_id = box.get_attribute("data-content-id")
        if not review_id or review_id in scraped_ids:
            continue

        # --- [별점 판별 로직] ---
        star_elements = box.find_elements(By.CSS_SELECTOR, "span[class*='semi'],
span[class*='score']")

        # 별점 요소가 없으면 '별점 없음'으로 간주하여 5점으로 할당
        if not star_elements or not star_elements[0].text.strip():
            star_val = '5'
        else:
            star_text = star_elements[0].text.strip()
            star_val = star_text if star_text.isdigit() else '5'

        # --- [조기 종료 조건] ---
        # 낮은 점수 순 정렬에서 1~2점 수집 중 3점 이상의 리뷰가 나타나면 더 이상 수집 불필요
        if star_val in ['3', '4']:
            print(f"\n[알림] {star_val}점 리뷰 도달. 타겟 점수대 수집을 조기 종료합니다.")
            stop_loop = True
            break

        # --- [데이터 수집 및 전처리] ---
        # 별점 없음(5)과 불만족(1, 2) 리뷰만 추출
        if star_val in ['5', '1', '2']:

            # 1. 멀티미디어 포함 여부 확인
            photo_check = box.find_elements(By.CSS_SELECTOR, "div[class*='ExpandableImage'],
div[data-button-name='후기이미지']")
            is_photo = 1 if len(photo_check) > 0 else 0

            # 2. 텍스트 정제: 불필요한 태그/중복 문구 제거
            content_el = box.find_element(By.CSS_SELECTOR, "[data-button-name='후기내용']")
            raw_txt = driver.execute_script("return arguments[0].textContent",
content_el).strip()

            clean_raw = raw_txt.replace("...더보기", "").replace("더보기", "").strip()
            temp_txt = clean_raw.replace(" ", "").replace("\n", "").replace("\r", "")
            half_len = len(temp_txt) // 2

            # 문장 반복 오류(A문장 + A문장 합쳐짐)를 체크하여 1개 문장만 추출
            if half_len > 0 and temp_txt[:half_len] == temp_txt[half_len:]:
                txt = clean_raw[:len(clean_raw)//2].strip()
            else:
                txt = clean_raw
            txt = ' '.join(txt.split())

            # 3. 메타데이터(날짜, 도움수) 추출
            date_val = box.find_element(By.CSS_SELECTOR, "
[class*='UserProfileSection__PurchaseDate']").text.strip()

            try:
                help_text = box.find_element(By.CSS_SELECTOR, "
[class*='HelpButton__Button']").text
                help_val = re.search(r'\d+', help_text).group(0) if re.search(r'\d+', help_text)
            else "0"

            except:
                help_val = "0"

            # 4. 추가 정보 (옵션/채형/만족도) 파싱
            g_option, body_info, satisfaction = "정보없음", "정보없음", "정보없음"
            option_rows = box.find_elements(By.CSS_SELECTOR,
"div[class*='OptionRow__Container']")
            for row in option_rows:
                # span:first-child는 항목명(라벨), last-child는 실제값
                label = row.find_element(By.CSS_SELECTOR, "span:first-
child").get_attribute("textContent").strip()
                value = row.find_element(By.CSS_SELECTOR, "span:last-
child").get_attribute("textContent").strip()
                if "옵션" in label: g_option = value
                elif "채형" in label: body_info = value
                elif "만족" in label: satisfaction = value

            # 추출 완료된 데이터를 파일에 즉시 쓰기 (메모리 보호)
            writer.writerow([p_id, date_val, star_val, is_photo, txt, g_option, body_info,
satisfaction, help_val])

            scraped_ids.add(review_id)
            scraped_count += 1
            new_data_found = True

            if scraped_count % 20 == 0:
                print(f"현재 {scraped_count}개 수집 완료...")

            except Exception:
                continue

    if stop_loop:
        break

```

```

            # 문장 반복 오류(A문장 + A문장 합쳐짐)를 체크하여 1개 문장만 추출
            if half_len > 0 and temp_txt[:half_len] == temp_txt[half_len:]:
                txt = clean_raw[:len(clean_raw)//2].strip()
            else:
                txt = clean_raw
            txt = ' '.join(txt.split())

            # 3. 메타데이터(날짜, 도움수) 추출
            date_val = box.find_element(By.CSS_SELECTOR, "
[class*='UserProfileSection__PurchaseDate']").text.strip()

            try:
                help_text = box.find_element(By.CSS_SELECTOR, "
[class*='HelpButton__Button']").text
                help_val = re.search(r'\d+', help_text).group(0) if re.search(r'\d+', help_text)
            else "0"

            except:
                help_val = "0"

            # 4. 추가 정보 (옵션/채형/만족도) 파싱
            g_option, body_info, satisfaction = "정보없음", "정보없음", "정보없음"
            option_rows = box.find_elements(By.CSS_SELECTOR,
"div[class*='OptionRow__Container']")
            for row in option_rows:
                # span:first-child는 항목명(라벨), last-child는 실제값
                label = row.find_element(By.CSS_SELECTOR, "span:first-
child").get_attribute("textContent").strip()
                value = row.find_element(By.CSS_SELECTOR, "span:last-
child").get_attribute("textContent").strip()
                if "옵션" in label: g_option = value
                elif "채형" in label: body_info = value
                elif "만족" in label: satisfaction = value

            # 추출 완료된 데이터를 파일에 즉시 쓰기 (메모리 보호)
            writer.writerow([p_id, date_val, star_val, is_photo, txt, g_option, body_info,
satisfaction, help_val])

            scraped_ids.add(review_id)
            scraped_count += 1
            new_data_found = True

            if scraped_count % 20 == 0:
                print(f"현재 {scraped_count}개 수집 완료...")

            except Exception:
                continue

    if stop_loop:
        break

```


• high rating 코드

데이터 추출 비율 low : high을 반영해
low rating data 추출량의 4배만큼을 MAX_SAVE_COUNT로 지정



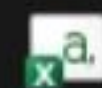
```
# --- [수집 설정 및 환경 변수] ---  
MAX_SAVE_COUNT = 200 # 추출하고자 하는 목표 리뷰 개수  
P_ID = "2723763"      # 무신사 상품 고유 번호 (URL에서 추출)  
TEST_PRINT_LIMIT = 50 # 실시간 수집 확인을 위한 터미널 출력 제한 개수  
SAVE_PATH = r"C:\Users\son76\Documents\2026_4학년1학기\kuggle_2026"
```



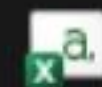
```
# 1. 대상 상품 리뷰 페이지 접속 (기본 정렬: 유용한순)  
driver.get(f"https://www.musinsa.com/review/goods/{P_ID}")  
wait = WebDriverWait(driver, 10)  
# 리뷰 아이템이 로드될 때까지 최대 10초 대기 (Explicit Wait)  
wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "div[class*='review-list-item']")))
```

• 결과

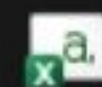
후기순 정렬기준 상위 6개 제품에 대하여 1:4 비율로 low rating,
high rating 크롤링 추출 완성



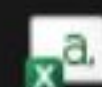
musinsa_low_1558847_final



musinsa_low_1954316_final



musinsa_low_2396645_final



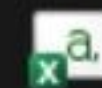
musinsa_low_2723764_final



musinsa_low_2723767_final



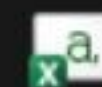
musinsa_low_5453617_final



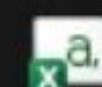
musinsa_high_1558847_final



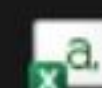
musinsa_high_1954316_final



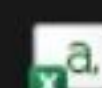
musinsa_high_2396645_final



musinsa_high_2723764_final



musinsa_high_2723767_final



musinsa_high_5453617_final

데이터 병합

• low / high rating file 병합 후 저장

- low data 중 별점 미작성 데이터 → 긍/부정 그룹화 불가로 인해 삭제
- 긍/부정 그룹화 → is_positive 변수 생성 후 1,0으로 인코딩

• 결측치 처리 및 제거

- purchase_option: 1개 삭제
- body_info : gender + height + weight 로 분할
'정보없음' : 578건 삭제
세 개의 값 중 하나라도 없는 데이터(불완전 데이터): 38건 삭제
- satisfaction: 결측치 3,275개를 삭제하긴 어려워 유지

[데이터 병합 결과]

항목	개수
불러온 high rating 파일 수	6개
불러온 low rating 파일 수	6개
총데이터개수	4,480건

[긍정(1) / 부정(0) 분포]

is_positive	개수
1	3,580건
0	900건
총데이터개수	4,480건

```
1. column 별 결측치 개수
product_id      0
date            0
rating          0
is_photo        0
review_text     0
purchase_option 0
body_info       0
satisfaction    0
helpful_indicator 0
is_positive     0
dtype: int64
```

```
2. '정보없음' 개수
purchase_option: 1건
body_info: 578건
satisfaction: 3275건
```

```
[body_info 처리 결과]
원본 데이터: 4480건
제거된 결측치 및 불완전 데이터: 616건
처리 후 데이터: 3864건
```


데이터 전처리



중복 데이터 처리

- review_text가 동일하지만, product_id에 따라 rating이 다른 데이터 존재
- rating이 다르지만 긍/부정 그룹에서는 벗어나지 않음.
중복 데이터 중 하나 씩만 유지. 나머지는 제거

가비지 리뷰 처리

- 10자 미만 리뷰 삭제 (리뷰 이벤트 조건이 20자 이상이므로 2번 반복 예상하여 10자 미만 필터링)
- 가비지 비율 (고유글자/전체글자)이 낮은 데이터 삭제

중복 데이터 삭제 후 데이터 개수: 3442건

10자 미만 리뷰
조회된 짧은 리뷰 개수: 1개

	review_text	rating
2793	기본템으로 좋아요	5

제거 후 현재 총 데이터 개수: 3440건

[현재 긍/부정 분포]

	is_positive
1	2848
0	592

Name: count, dtype: int64

• 특수문자 등 불필요한 기호 처리

- HTML 태그, 이메일 등 정규표현식으로 정제

ㅏ나 ㄴㄴ 과 같이 부정적 표현이 삭제되는 것을 막기 위해 '특수문자'는 유지.

- date 값 일관되게 표현

26.5.6을 기준으로 설정.

date열의 문자 데이터를 포함해서 모두 날짜 데이터로 변환.

- 맞춤법 및 띄어쓰기 수정

실제 유효한 변화는 없다 생각됨.

‘되어 → 돼’와 같은 정석적인 오타 수정, 그러나 ‘종른 → 좋은’과 같은 의미적 오타는 수정 불가.

내용이 변경된 데이터: 0건
변경사항이 있는 데이터가 없습니다.

변환 결과

	date_before	date
0	26.04.04	2026-04-04
1	26.04.03	2026-04-03
2	26.04.03	2026-04-03
3	26.04.02	2026-04-02
4	26.02.27	2026-02-27
5	26.02.23	2026-02-23
6	26.02.19	2026-02-19
7	26.02.09	2026-02-09
8	26.01.26	2026-01-26
9	25.12.18	2025-12-18

=== 전체 데이터 정제 시작 ===
100%|██████████| 3440/3440 [00:14<00:00, 239.97it/s]

=== 전체 데이터 정제 완료 ===
검사한 데이터: 3440건
실제 교정된 데이터: 383건
[실제 교정 사례 (Before -> After)]

Before: 일단 두개를 주문했는데 배송패키지보고 1차충격.. 부피가 있는데 박스포장이 아닌 구역구역 맞지도 않은 택배봉투에 쑤셔넣어서 배송했더라구요? 심지어 옆에 찢겨왔습니다 뜯는데도 풀리 찢길까봐 애초 먹을정도로 포장상태가 일단 너무 충격이었습니다.. 옷 포장 풀리는 이미 누가 개봉한거같은 무지 풀리가 오픈되어 테이핑이 붙어있다 말았더군요.. 일단 두개를 주문했는데 배송패키지보고 1차충격.. 부피가 있는데 박스포장이 아닌 구역구역 맞지도 않은 택배봉투에 쑤셔넣어서 배송했더라구요? 심지어 옆에 찢겨왔습니다 뜯는데도 풀리 찢길까봐 애초 먹을정도로 포장상태가 일단 너무 충격이었습니다.. 옷 포장 풀리는 이미 누가 개봉한거같은 무지 풀리가 오픈되어 테이핑이 붙어있다 말았더군요..

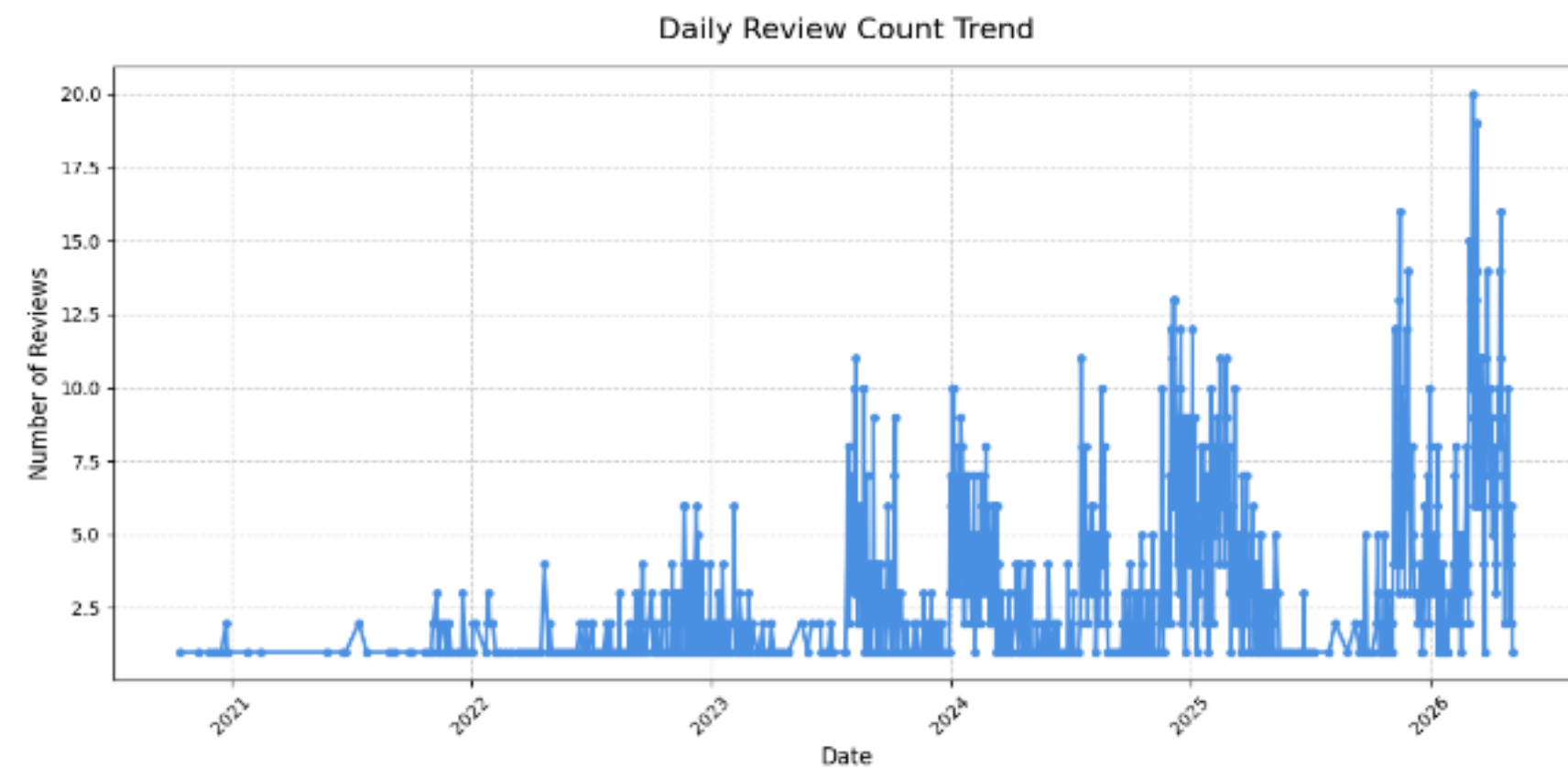
After: 일단 두개를 주문했는데 배송패키지보고 1차충격.. 부피가 있는데 박스포장이 아닌 구역구역 맞지도 않은 택배봉투에 쑤셔넣어서 배송했더라구요? 심지어 옆에 찢겨왔습니다 뜯는데도 풀리 찢길까봐 애초 먹을정도로 포장상태가 일단 너무 충격이었습니다.. 옷 포장 풀리는 이미 누가 개봉한거같은 무지 풀리가 오픈돼 테이핑이 붙어있다 말았더군요.. 일단 두개를 주문했는데 배송패키지보고 1차충격.. 부피가 있는데 박스포장이 아닌 구역구역 맞지도 않은 택배봉투에 쑤셔넣어서 배송했더라구요? 심지어 옆에 찢겨왔습니다 뜯는데도 풀리 찢길까봐 애초 먹을정도로 포장상태가 일단 너무 충격이었습니다.. 옷 포장 풀리는 이미 누가 개봉한거같은 무지 풀리가 오픈돼 테이핑이 붙어있다 말았더군요..

Before: 몇 번 입으니까 보풀 장난 아닙니다 가격이 저렴하더라도 일회용 옷도 아니고 이거 입어라는건지 말라는건지 참
After: 몇 번 입으니까 보풀 장난 아닙니다 가격이 저렴하더라도 일회용 옷도 아니고 이거 입어라는거인지 말라는거인지 참

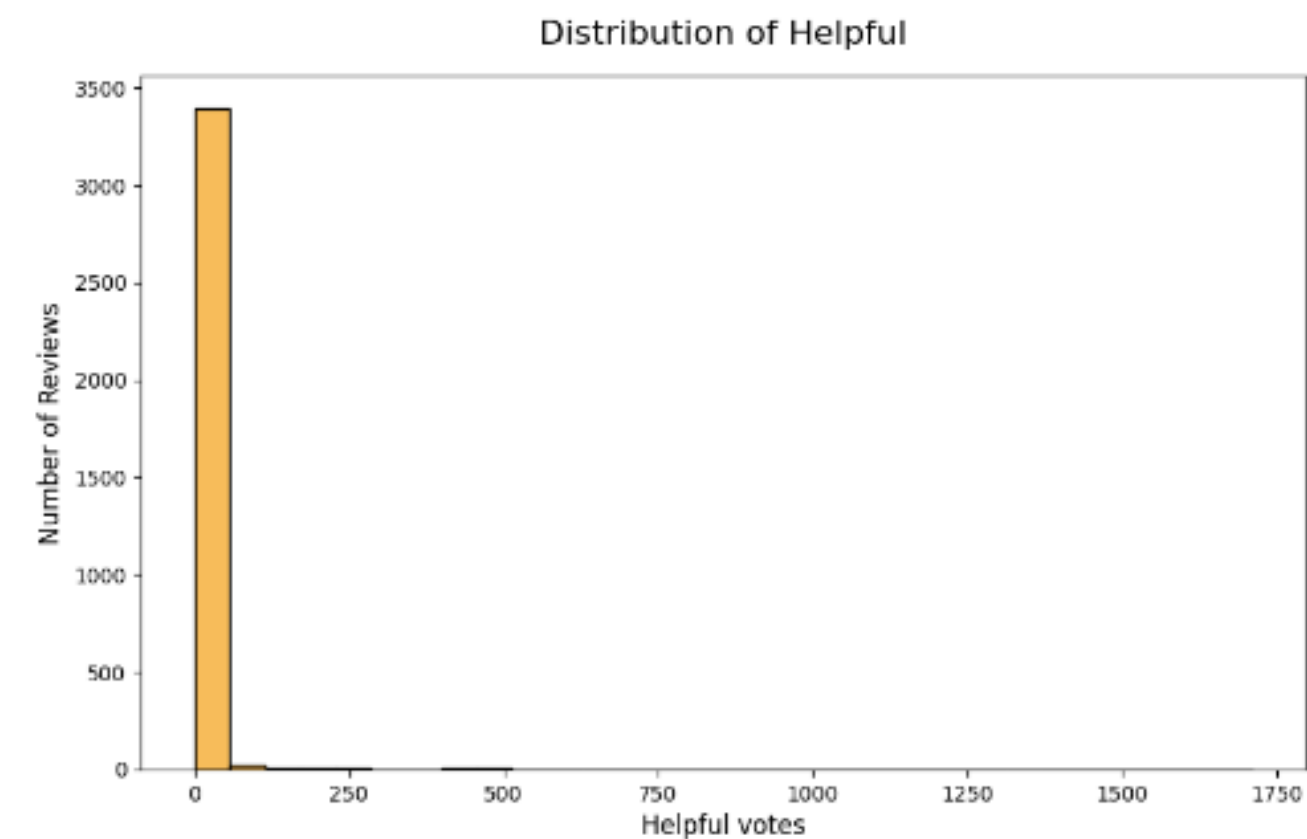
전처리 후 데이터 시각화



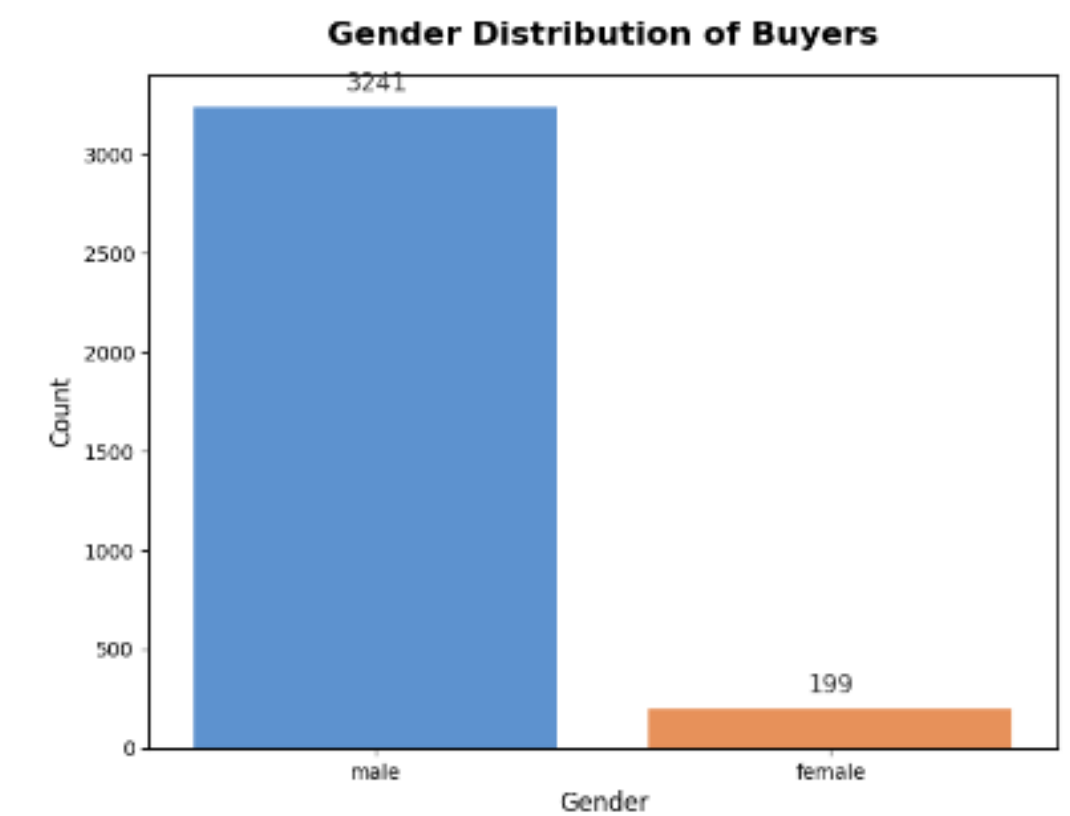
리뷰 작성일 분포



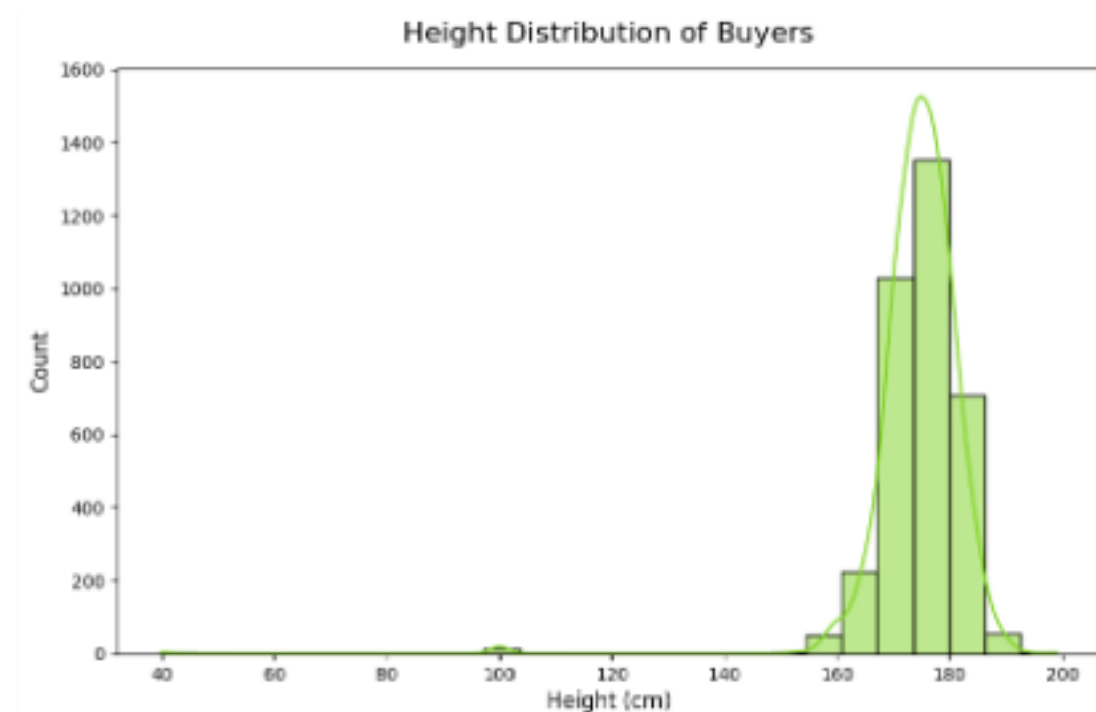
'도움돼요' 분포



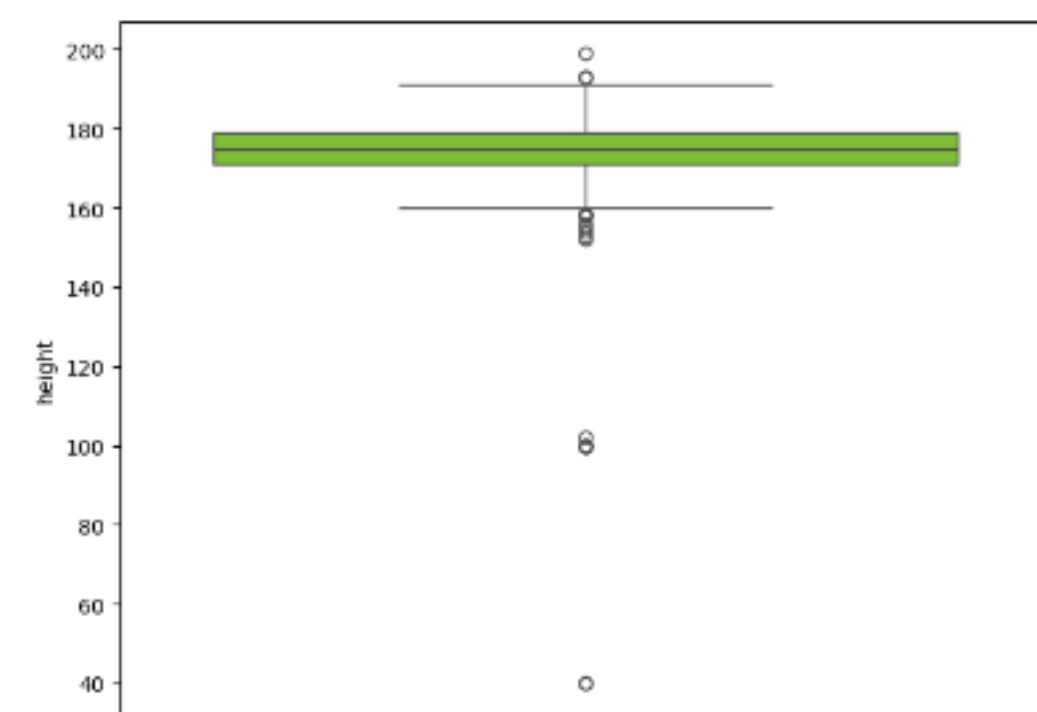
gender



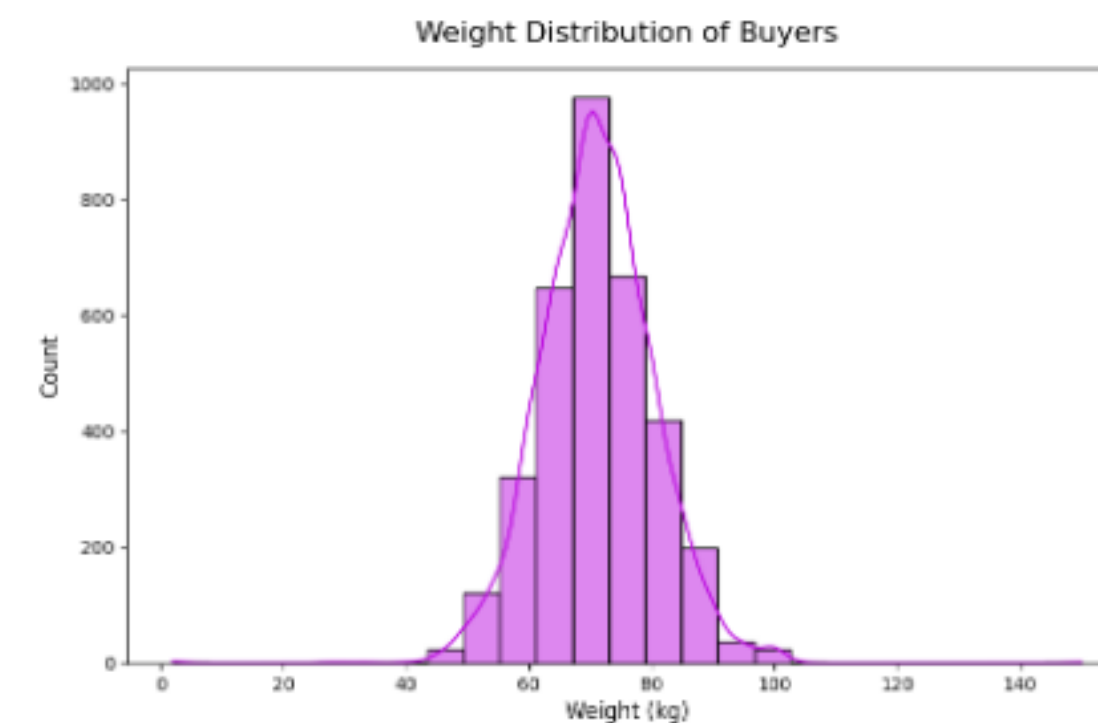
height_histogram



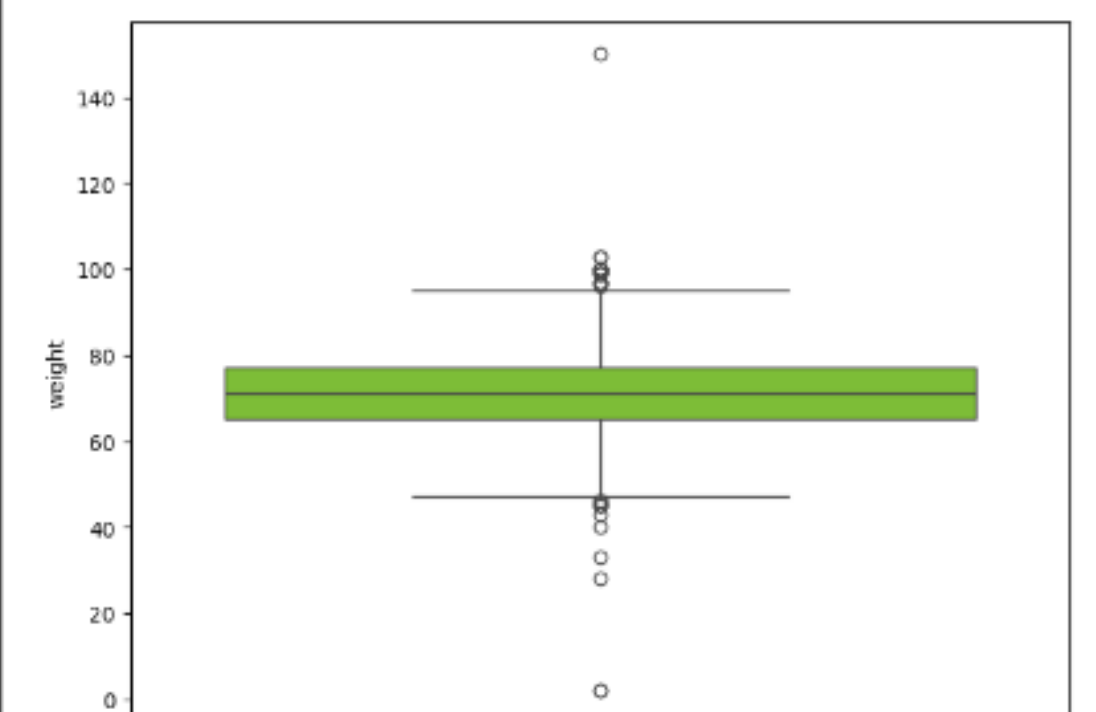
height_boxplot



weight_histogram



weight_boxplot



이상치 제거 및 데이터 불균형 대응

- 이상치 구간 (IQR*1.5), 극도의 이상치만 제거
 - height: 14건
 - weight: 4건

- 긍/부정 데이터 불균형 대응
 - Under / Over Sampling

	product_id	date	rating	is_photo	helpful_indicator	is_positive	height	weight
count	3.424000e+03	3424	3424.000000	3424.000000	3424.000000	3424.000000	3424.000000	3424.000000
mean	2.243903e+06	2024-10-23 17:23:49.906542080	4.309871	0.877336	4.508178	0.827979	175.039720	71.168224
min	1.558847e+06	2020-10-15 00:00:00	1.000000	0.000000	0.000000	0.000000	152.000000	40.000000
25%	1.558847e+06	2024-01-18 00:00:00	4.000000	1.000000	0.000000	1.000000	171.000000	65.000000
50%	1.954316e+06	2024-12-23 00:00:00	5.000000	1.000000	0.000000	1.000000	175.000000	71.000000
75%	2.723764e+06	2025-11-30 00:00:00	5.000000	1.000000	0.000000	1.000000	179.000000	77.000000
max	5.453617e+06	2026-05-06 00:00:00	5.000000	1.000000	1711.000000	1.000000	199.000000	103.000000
std	8.972490e+05	NaN	1.321989	0.328099	46.507325	0.377454	5.641148	8.935924

원본 데이터 개수: 긍정(2835건), 부정(589건)
1. Under sampling 후 데이터 개수: 총 1178건
2. Over sampling 후 데이터 개수: 총 5670건

데이터 탐색 및 키워드 추출



· 부정 데이터셋에서 주로 등장한 구문 / 단어들의 출현 빈도를 계산하는 코드를 실행

```
# 1. 전처리된 데이터 로드 (결측치 제거)
df = df.dropna(subset=['review_text'])

# 2. 긍정(4~5점) / 부정(1~2점) 데이터 분리
pos_reviews = df[df['rating'] >= 4]['review_text'].tolist()
neg_reviews = df[df['rating'] <= 2]['review_text'].tolist()

# 3. CountVectorizer 초기화 (N-gram 적용)
# ngram_range=(1, 2): 단어 1개짜리와 2개짜리 조합을 모두 찾음 (예: '보통', '보통 심함')
# min_df=3: 전체 문서에서 최소 3번 이상 등장한 단어만 취급 (노이즈 제거)
vectorizer = CountVectorizer(ngram_range=(1, 2), min_df=3)

# 4. 전체 데이터로 단어장(Vocabulary) 학습 및 빈도 추출
# 긍정과 부정 리뷰들 합쳐서 벡터화합니다.
X_pos = vectorizer.fit_transform(pos_reviews)
X_neg = vectorizer.transform(neg_reviews)

# 단어 목록 가져오기
feature_names = vectorizer.get_feature_names_out()

# 5. 긍정/부정 리뷰 각각에서 단어별 총 출현 빈도 계산
pos_sum = X_pos.sum(axis=0).A1
neg_sum = X_neg.sum(axis=0).A1

# 데이터프레임으로 정리
word_freq_df = pd.DataFrame({
    'word': feature_names,
    'pos_count': pos_sum,
    'neg_count': neg_sum
})
```

```
# 6. '부정 편향성(Negative Score)' 계산
# 전체 출현 횟수(긍정+부정) 중 부정 리뷰에서 출현한 비율을 계산합니다.
# 데이터가 너무 적은 단어는 제외하기 위해 전체 빈도 5회 이상인 단어만 필터링
word_freq_df['total_count'] = word_freq_df['pos_count'] +
word_freq_df['neg_count']
filtered_df = word_freq_df[word_freq_df['total_count'] >= 5].copy()

# 부정 비율 = (부정 빈도) / (전체 빈도)
filtered_df['neg_ratio'] = filtered_df['neg_count'] /
filtered_df['total_count']

# 7. 부정 비율이 높은 순으로 정렬 (1.0에 가까울수록 부정 리뷰에만 등장하는 단어)
# 긍정 리뷰보다 부정 리뷰에서 더 많이 쓰인 단어(neg_ratio > 0.5) 추출
negative_keywords = filtered_df[filtered_df['neg_ratio'] >
0.5].sort_values(by=['neg_ratio', 'neg_count'], ascending=[False, False])

print("데이터 기반 100% 리얼 부정 특화 키워드/구문 Top 20")
print(negative_keywords[['word', 'neg_count', 'pos_count',
'neg_ratio']].head(20))
```

· 분석 기법: CountVectorizer 활용

N-gram: 1 ~ 2, 최소 출현 빈도: 3회 이상

· 핵심 로직(Negative Score):

단어별 전체 출현 빈도 중 부정 리뷰에서 출현한 비율을 계산하여 부정 편향성이 높은 단어 도출.

(ex: 부정 비율이 1.0에 가까울수록 부정 리뷰에만 등장)

· 데이터 기반 100% 리얼 부정 특화키워드 / 구문 Top 20

Word	neg_count	pos_count	neg_ratio
보풀이 너무	36	5	0.878049
한번	58	15	0.794521
hellip	17	5	0.772727
몇번	18	6	0.750000
괜찮은데	12	4	0.750000
보고	8	3	0.727273
안입고	8	3	0.727273
입었는데	29	13	0.690476
생겨서	11	5	0.687500
보풀	55	26	0.679012
이렇게	19	9	0.678571
않을	6	3	0.666667
크네요	6	3	0.666667
하면	7	4	0.636364
옷을	19	11	0.633333
그냥	67	40	0.626168
나요	5	3	0.625000
납니다	5	3	0.625000
내구성이	5	3	0.625000
디자인은	5	3	0.625000

· Top 20 데이터 해독 및 피처화 전략

1. 쓰레기 데이터 (정제 대상)

hellip (77%): 웹사이트에서 쓰는 말줄임표의 흔적

→ 피처로 쓰는 게 아니라, 1단계 전처리 코드의 특수문자 제거 정규식에 추가해서 지워야 함.

2. 진짜 핵심 불만 (Core Issue)

보풀이 너무 (87%), 보풀 (67%), 생겨서 (68%), 내구성이 (62%):
이 니트의 최대 단점은 압도적으로 '보풀'인 것을 알 수 있음. "보풀이 너무 많이 생겨서.." 라는 문장이
고객들의 단골 멘트임을 파악.

→ Issue_Durability 피처에 강력한 가중치를 두어야 함.

3. 조건부 불만 (Tlme/Action Context) - 중요

한번 (79%), 몇번 (75%), 입었는데 (69%), 안입고 (72%):
고객들은 받자마자 불만을 터뜨리지 않는다. "한번 입었는데", "몇번 안입고 빨았더니" 보풀이 생기거나 망가진
다는 뜻.

→ 옷을 입거나 세탁한 '이후'에 발생하는 품질 문제를 잡아내는 Issue_After_Wear (착용 후 변형) 이라는 새로
운 피처를 만들어야 함.

4. 칭찬을 가장한 치명적 불만 (But)

괜찮은데 (75%), 디자인은 (62%):
"디자인은 예쁜데 보풀이 심해요". 즉, 긍정적인 단어로 시작하되 결국 낮은 별점을 준 리뷰들이다.
이는 단순 감성 사전이 가장 헛갈려하는 문장이다.

→ Mixed_Sentiment (아쉬움/기대이하) 라는 피처를 만들어 이런 패턴을 분류.



```
def generate_data_driven_features(df):
    text = df['review_text'].astype(str)

    # 1. 데이터 기반 불만 피쳐 (Top 20 인사이트 반영)
    # [핵심] 보풀 및 내구성 불만 (보풀, 생겨서, 내구성 반영)
    df['Issue_Pilling'] = text.apply(
        lambda x: 1 if re.search(r'보풀|먼지|내구성.*(별로|약해|안좋)|생겨서', x) else 0
    )

    # [발견] 착용/세탁 후 발생한 품질 불만 (한번, 몇번, 입었는데 반영)
    df['Issue_After_Wear'] = text.apply(
        lambda x: 1 if re.search(r'(한번|한 번|몇번|몇 번|하루).*(입었|입고|빨았|세탁|입자마자)', x) else 0
    )

    # [발견] 부분적 만족, 그러나 치명적 단점 존재 (괜찮은데, 디자인은 반영)
    df['Mixed_Sentiment_But'] = text.apply(
        lambda x: 1 if re.search(r'(디자인은|핏은|색감은|괜찮은데|예쁜데).*(근데|하지만|보풀|따갑|별로|아쉽)',
x) else 0
    )

    # 사이즈 불만 (크네요 반영)
    df['Issue_Size'] = text.apply(
        lambda x: 1 if re.search(r'크네요|너무 커|어병|작네요|작아', x) else 0
    )

    # 2. 도메인 기본 불만 & 메타 피쳐 (기존 유지)
    # 촉감 불만 (부정어 방어 로직 유지)
    df['Issue_Touch'] = text.apply(
        lambda x: 1 if re.search(r'(?<!안 )(?!않)(?!못)(?!덜 ).*(까슬|따갑|거칠|겉고|간지러)', x) else 0
    )

    # 리뷰 길이
    df['Meta_Review_Length'] = text.str.len()

    # 부정 강조 부사 포함 여부
    df['Meta_Negative_Adverb'] = text.apply(
        lambda x: 1 if re.search(r'너무|진짜|완전|정말|개', x) else 0
    )

    return df

# 피쳐 생성 적용
df_feature = generate_data_driven_features(df)
```

• Garbage Data 정제:

'hellip' 같은 웹사이트 HTML 엔티티 흔적 발견.

• Issue_Pilling (핵심 불만 - 내구성):

'보풀', '생겨서', '내구성이' 등 제품의 최대 단점인 보풀 관련 문구 매핑.

• Issue_After_Wear (조건부 불만 - 착용/세탁 후):

'한번', '몇번 안입고' 등 고객이 상품 수령 직후가 아닌, 특정 시점 이후에 느끼는 품질 불만 패턴 매핑.

• Mixed_Sentiment_But (혼합 감성 - 아쉬움):

"디자인은 예쁜데 보풀이 심해요"처럼 긍정 단어로 시작해 부정으로 끝나는 리뷰 분류 (단순 감성 사전의 한계 극복).

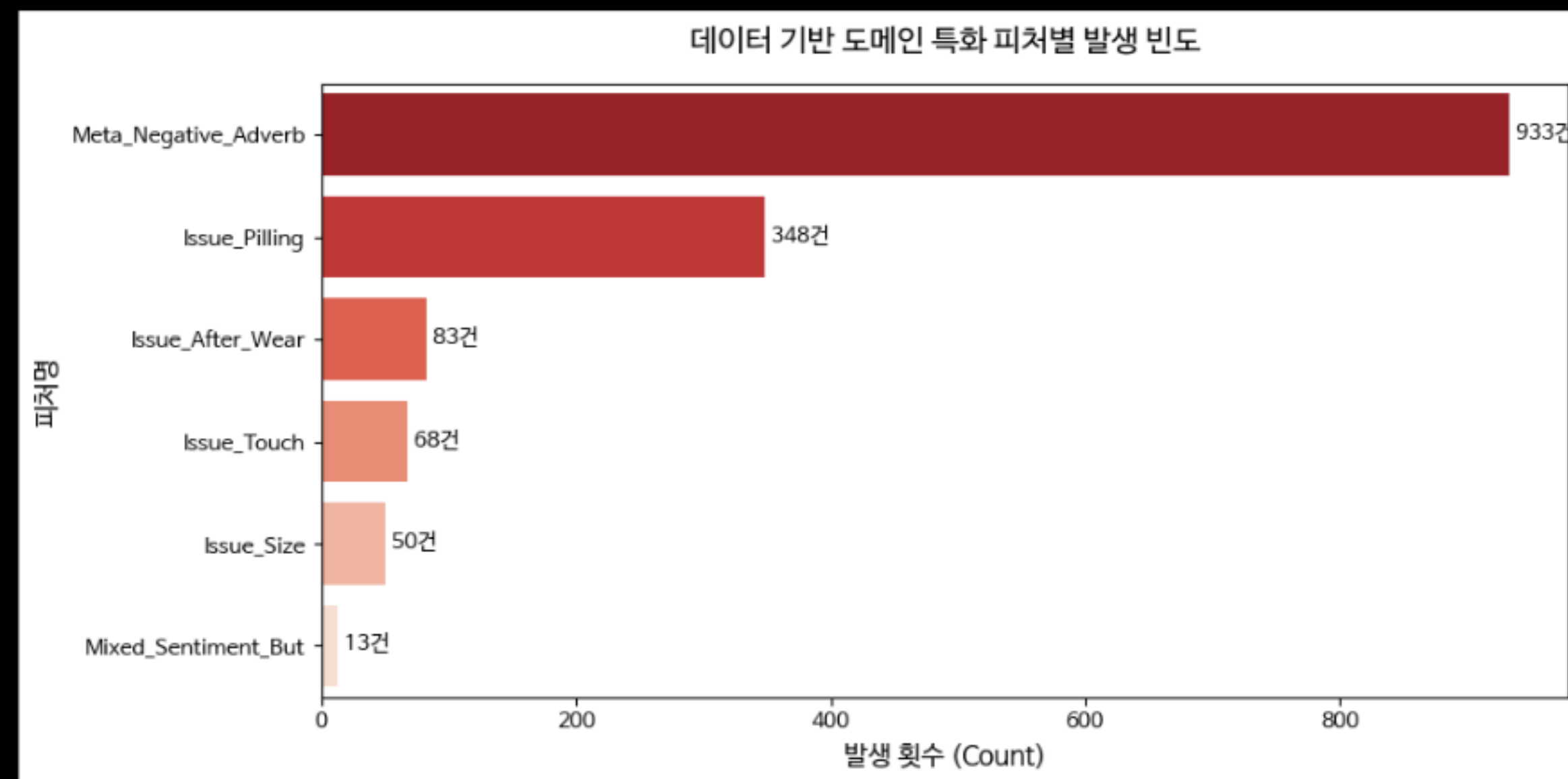
데이터 시각화



· 피처별 발생 빈도 (Bar Chart)

- 새롭게 생성한 도메인 특화 피처들이 전체 리뷰에서 얼마나 자주 발생하는지 건수와 비율(%)로 시각화.

→ 부정 부사 포함 여부가 27%로 가장 높고, 보풀 문제가 10%로 핵심 불만임을 증명.



--- 전체 데이터 대비 피처별 발생 비율 (%) ---

Issue_Pilling 10.16%

Issue_After_Wear 2.42%

Mixed_Sentiment_But 0.38%

Issue_Size 1.46%

Issue_Touch 1.99%

Meta_Negative_Adverb 27.25%

dtype: object

· 평점(Rating)과의 상관관계 (Heatmap)

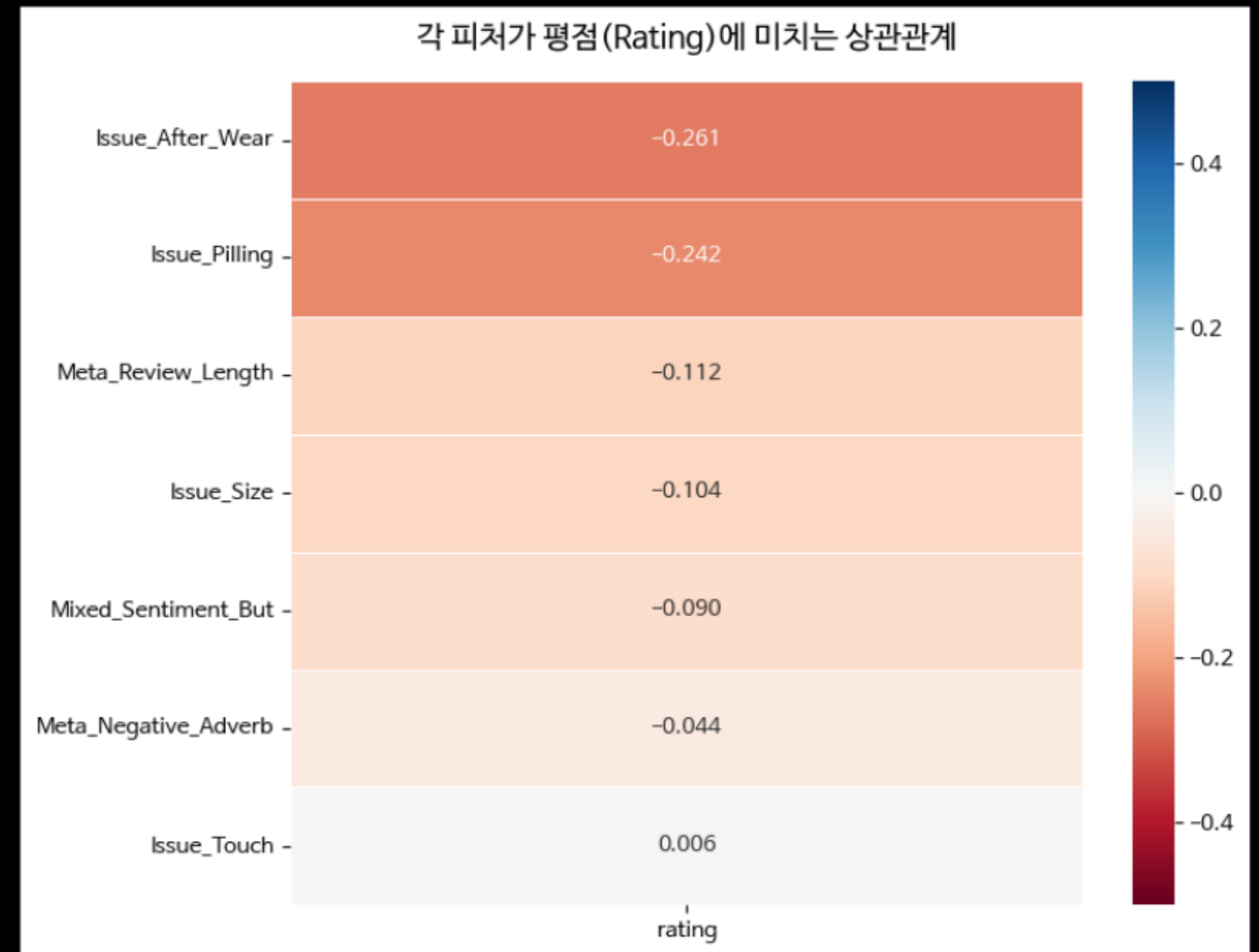
```
# 4. 평점(rating) 및 리뷰 길이(Meta_Review_Length)를 포함한 상관관계 분석
target_cols = ['rating', 'Meta_Review_Length'] + domain_features
corr_matrix = df_feature[target_cols].corr()

# 'rating'과의 상관관계수만 추출하여 정렬 (자기 자신인 rating 제외)
rating_corr = corr_matrix[['rating']].drop('rating').sort_values(by='rating',
                                                                    ascending=True)

plt.figure(figsize=(8, 6))
sns.heatmap(rating_corr, annot=True, cmap='RdBu', fmt='.3f', linewidths=0.5,
            vmin=-0.5, vmax=0.5)

plt.title('각 피처가 평점(Rating)에 미치는 상관관계', fontsize=14, pad=15)
plt.ylabel('')
plt.tight_layout()
plt.show()
```

- 복잡한 전체 상관관계 행렬 대신, 모델의 예측 타겟이 될 평점(Rating) 컬럼과의 상관관계수만 필터링하여 시각화.



- 어떤 피처가 별점에 가장 치명적인 악영향을 미치는지 직관적으로 리포팅.

타겟 변수 생성



- 고객이 남긴 1 ~ 5점의 별점을 '긍정(0)' 과 '불만(1)' 이라는 두 가지 선택지로 단순화.
- 머신러닝 모델에게는 '불만 리뷰인지 아닌지'라는 OX 퀴즈를 내는 것이 학습 효율이 훨씬 좋기 때문.
- 추가적으로 훈련/테스트 데이터를 분할하고 train 데이터로만 단어 사전을 학습하고 수치화해 데이터 누수를 방지.

```
총 리뷰 개수: 3424건
모델이 학습할 타겟(0: 긍정, 1: 부정)의 불균형 비율 확인
target
0      82.8 %
1      17.2 %
Name: proportion, dtype: object
```

머신러닝 모델 학습 및 파이프라인 생성



· Logistic Regression 모델 적용

- 선택 이유:

딥로지스틱 회귀는 결과에 대한 이유를 가장 투명하게 보여주는 '화이트박스(White-box)' 모델이므로, 프로젝트의 기준점(Baseline)을 잡기 위해 1순위로 선택.

- 해당 프로젝트에서의 장점:

단어(TF-IDF)와의 조합: 이 프로젝트 데이터는 1,000개의 단어로 쪼개져 있어 대부분이 '0'으로 채워진 희소 행렬(Sparse Matrix)이다.

직관적인 인사이트 도출: 각 단어와 파생 변수에 명확한 '가중치(Coefficient)'를 부여하므로, Issue_After_Wear 같은 변수가 실제로 작동한다는 것을 완벽하게 증명해 주었습니다.

· 인공지능 튜닝 도구 Optuna 적용

• 모델 평가 및 결과 시각화

최적의 하이퍼파라미터: {'C': 1.2970909857633575, 'solver': 'liblinear'}

최고 F1-Score (Train CV): 0.6836

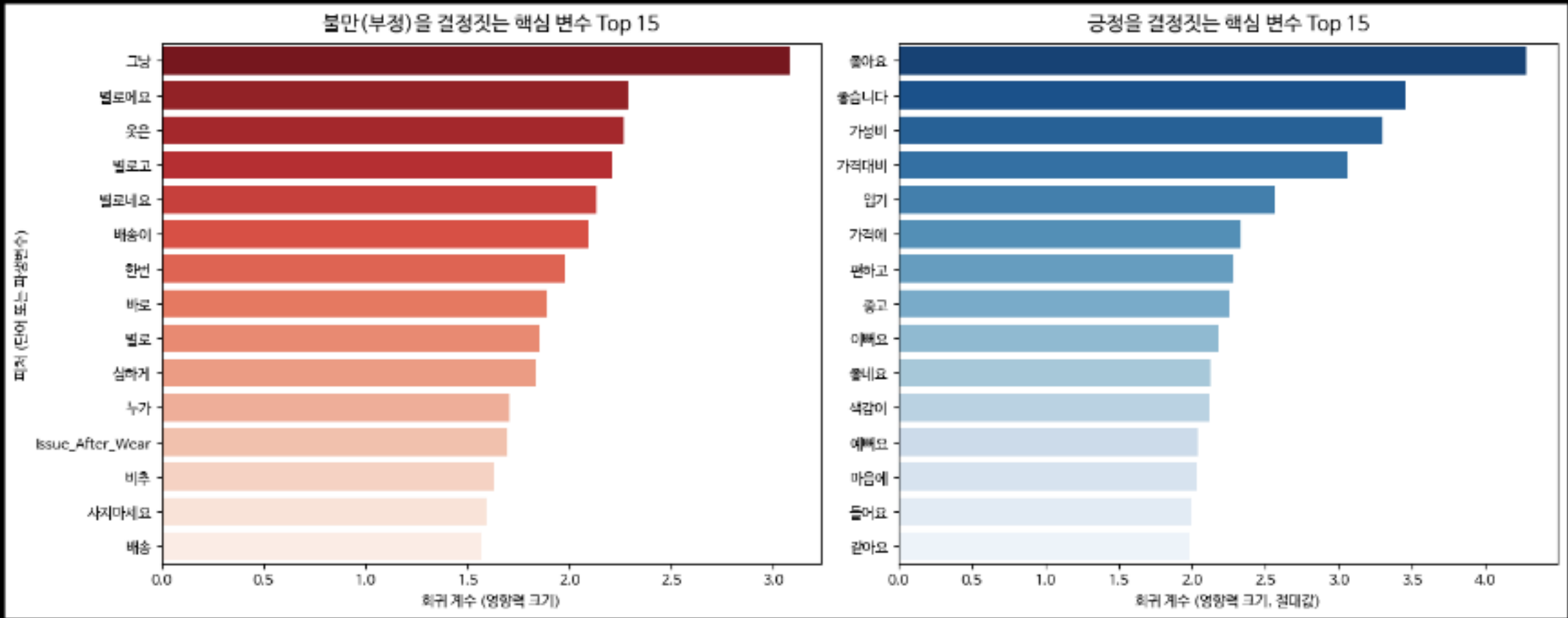
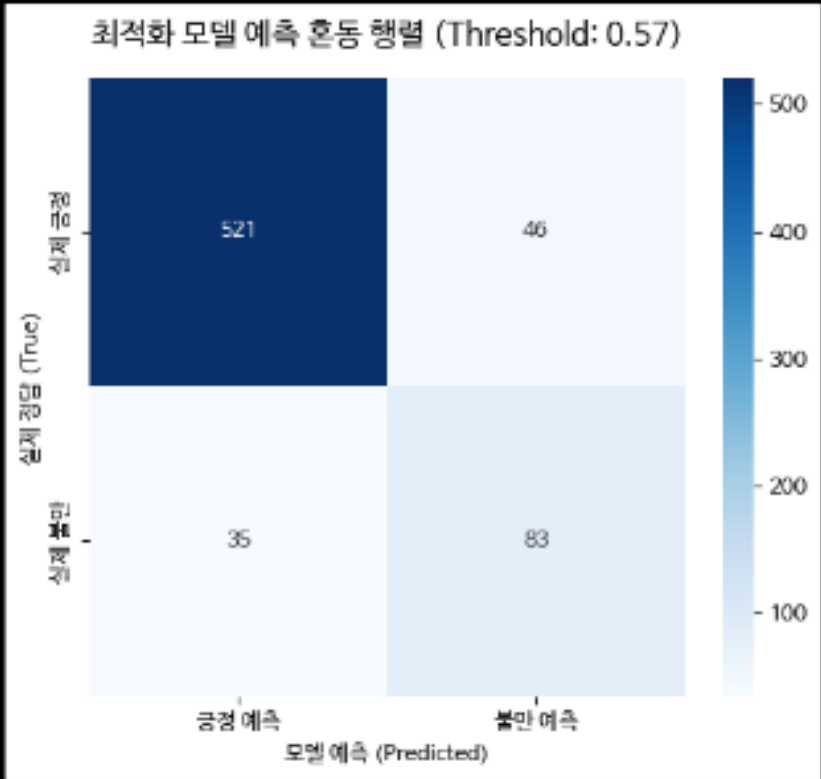
임계값 (Threshold) 최적화 탐색 중...

★ 최적의 불만 판정 임계값: 0.57

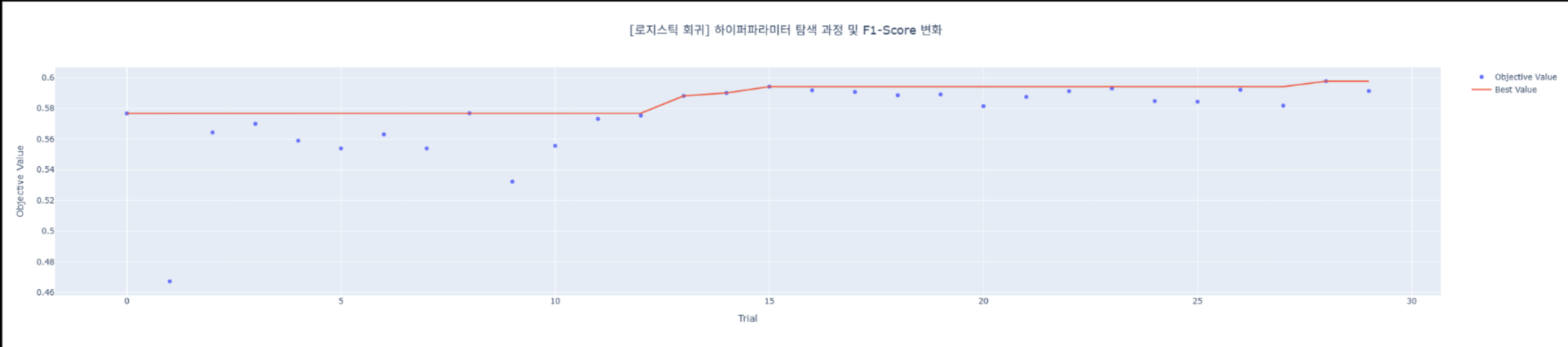
★ 최적 임계값 적용 시 F1-Score: 0.6721

[로지스틱 회귀 최적화 모델 평가 결과: Test Data]

	precision	recall	f1-score	support
긍정(0)	0.94	0.92	0.93	567
불만(1)	0.64	0.70	0.67	118
accuracy			0.88	685
macro avg	0.79	0.81	0.80	685
weighted avg	0.89	0.88	0.88	685



• Optuna 하이퍼파라미터 과정을 시각적으로 확인



• LightGBM 모델 적용

- 선택 이유:

로지스틱 회귀가 단어 하나하나의 점수를 단순히 더하는 방식이라면, 고객의 불만은 "A단어가 있고, 동시에 B단어가 없을 때"처럼 복잡한 조건(비선형 패턴)으로 나타날 수 있다. 이를 잡아내기 위해 오답을 집중적으로 고쳐나가며 학습하는 모델 적용.

- 해당 프로젝트에서의 장점:

자동 불균형 해소: 이 프로젝트 데이터는 긍정과 부정이 4:1로 심하게 불균형하다. LightGBM은 내부에 `is_unbalance=True`라는 파라미터가 있어서, 오버샘플링(SMOTE) 없이도 알고리즘 스스로 부정을 틀렸을 때 더 큰 벌점을 주어 불균형을 잘 잡아낸다.

빠른 학습 속도: 다른 트리 모델들보다 훨씬 가볍고 빠르게 학습하여 짧은 시간 안에 다양한 테스트를 해볼 수 있음.

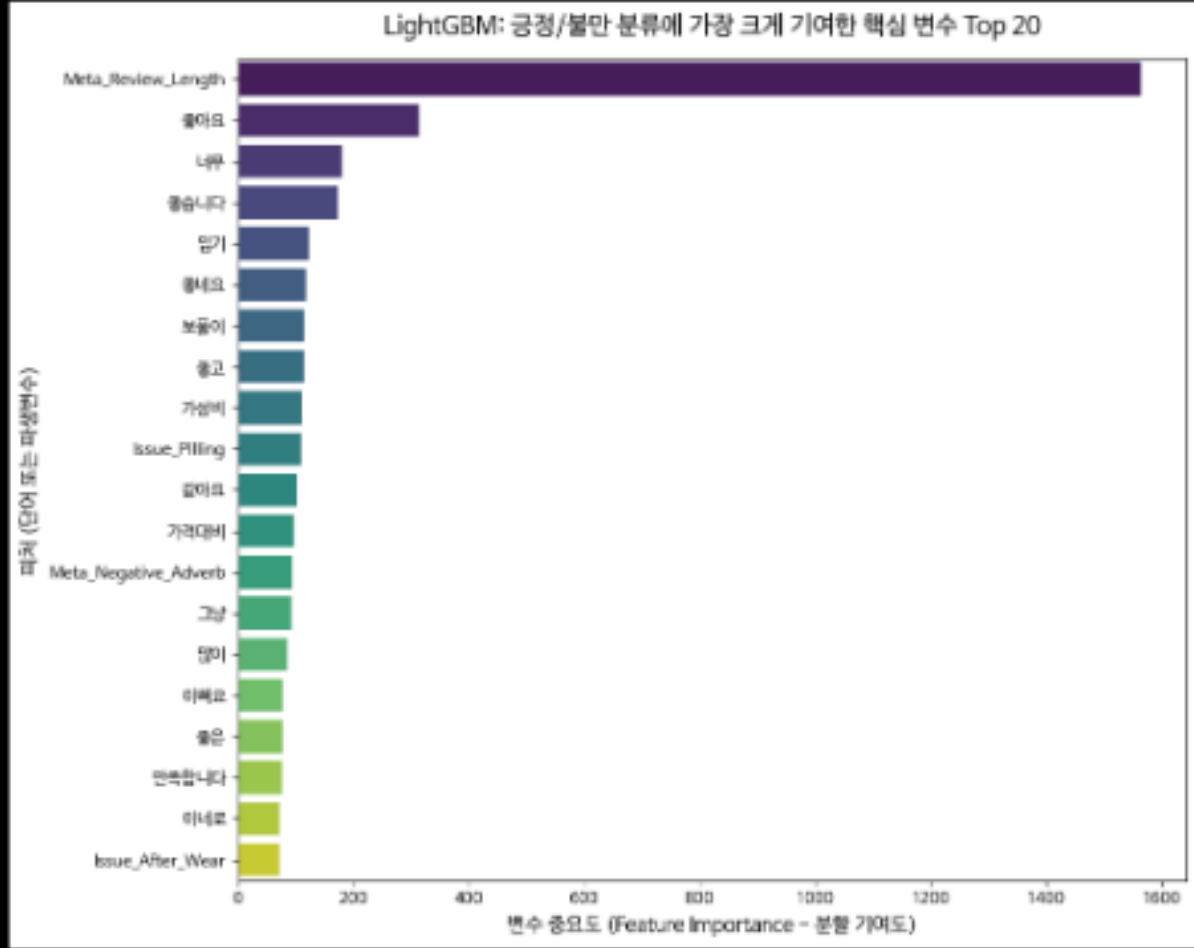
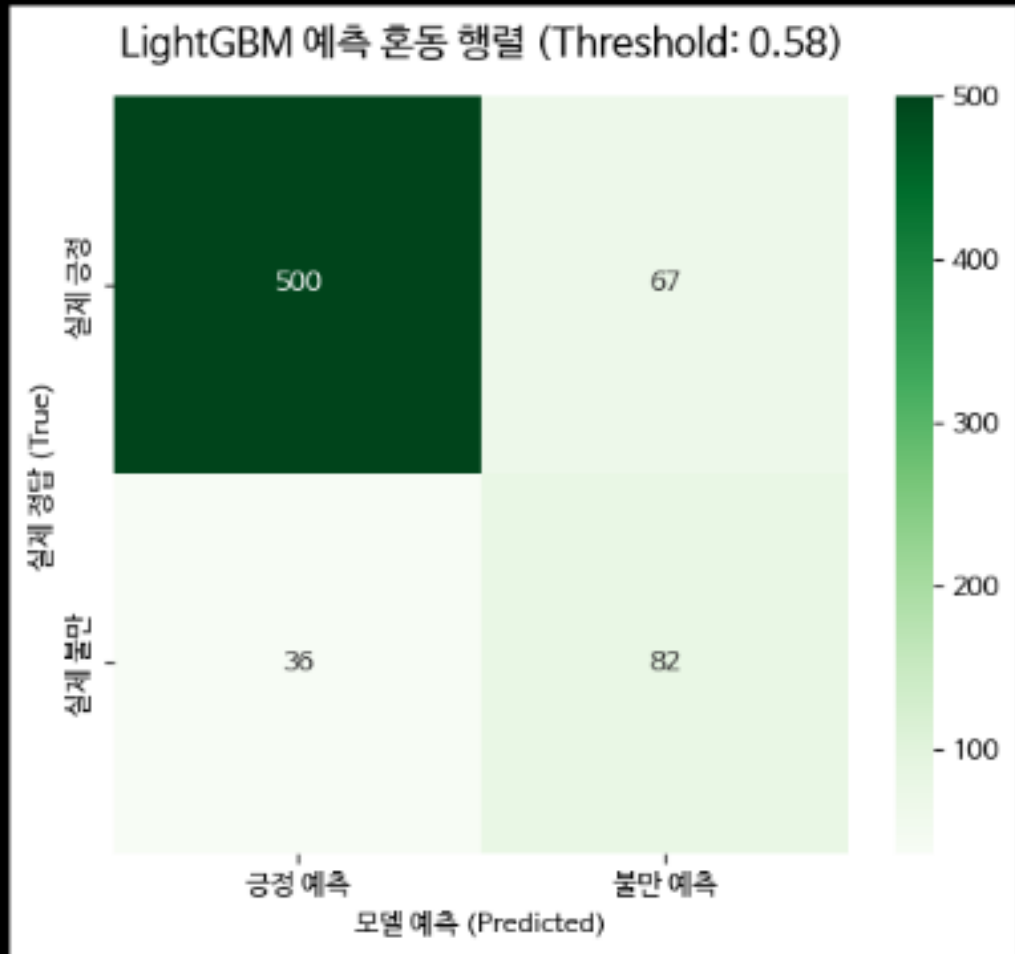
• 모델 평가 및 결과 시각화

임계값(Threshold) 최적화 탐색 중...

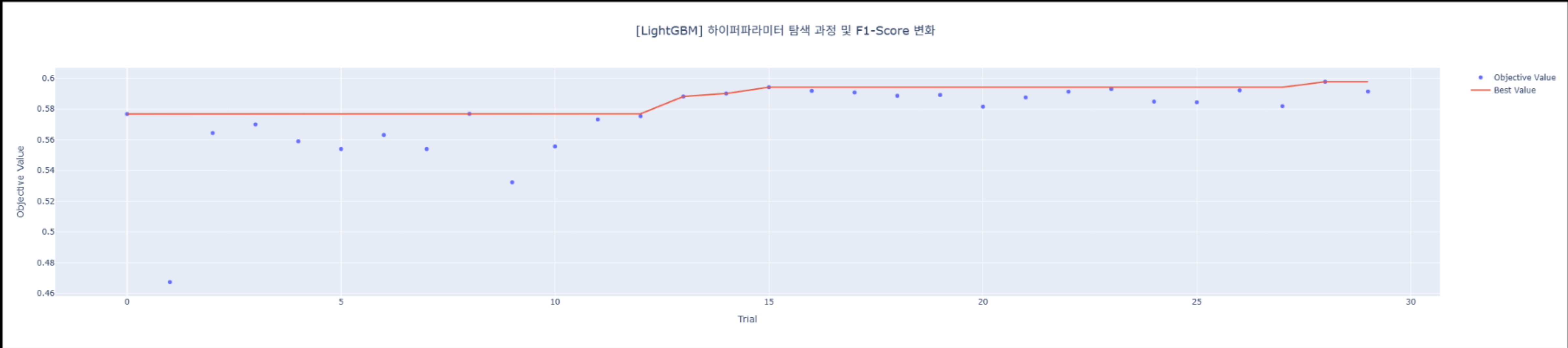
- ★ 최적의 불만 판정 임계값: 0.58
- ★ 최적 임계값 적용 시 F1-Score: 0.6142

[LightGBM 최적화 모델 평가 결과: Test Data]

	precision	recall	f1-score	support
긍정(0)	0.93	0.88	0.91	567
불만(1)	0.55	0.69	0.61	118
accuracy			0.85	685
macro avg	0.74	0.79	0.76	685
weighted avg	0.87	0.85	0.86	685



• Optuna 하이퍼파라미터 과정을 시각적으로 확인



• Random Forest 모델 적용

- 해당 프로젝트에서의 장점:

소규모 데이터에서의 강력한 방어력: 데이터가 적고 희소해도 예측 성능이 크게 널뛰지 않고 작동.

정교한 가중치 적용: `class_weight='balanced_subsample'` 기능을 통해, 수백 개의 나무를 기를 때마다 매번 긍정/부정 비율을 새로 계산해서 가중치를 준다.

• 모델 평가 및 결과 시각화

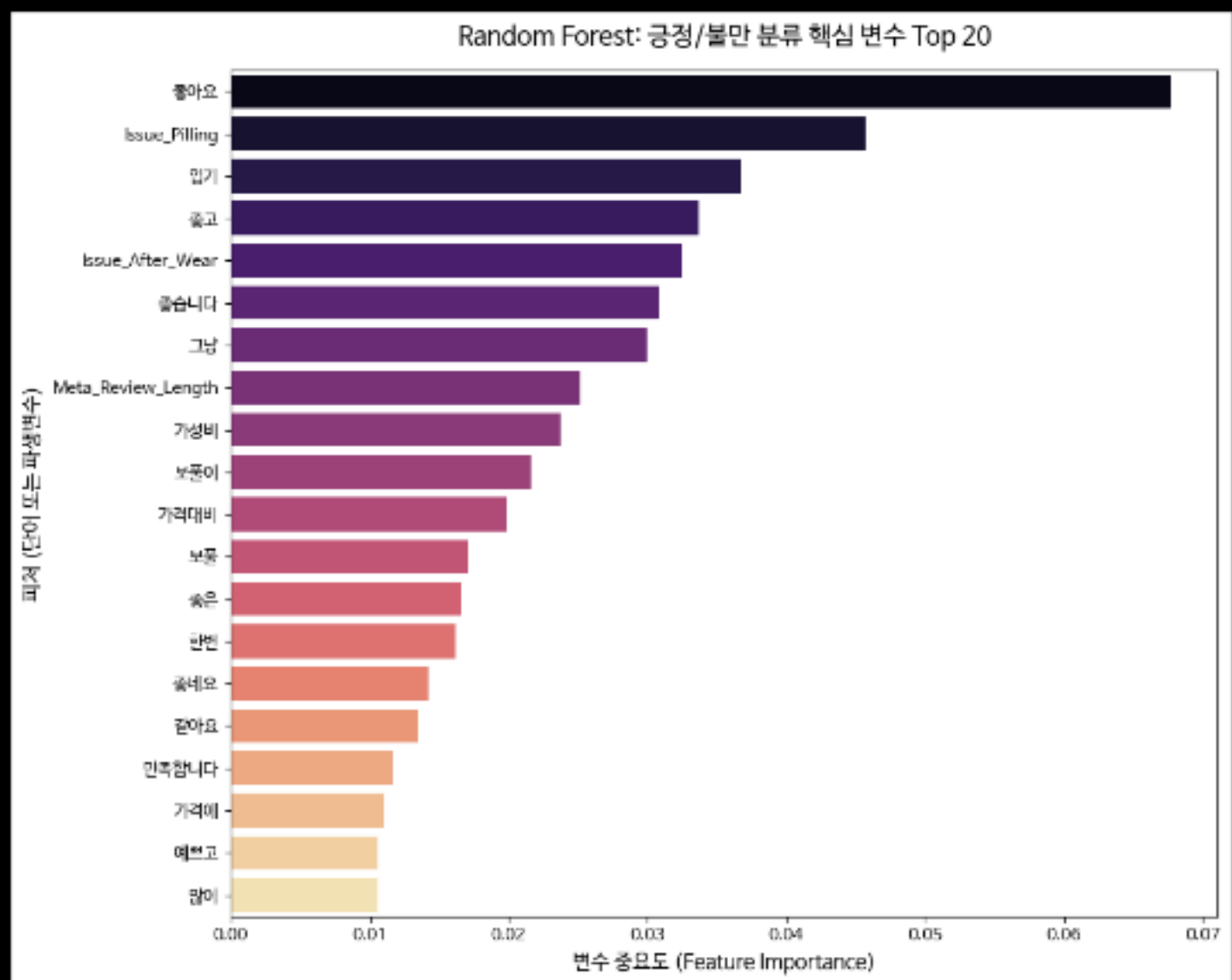
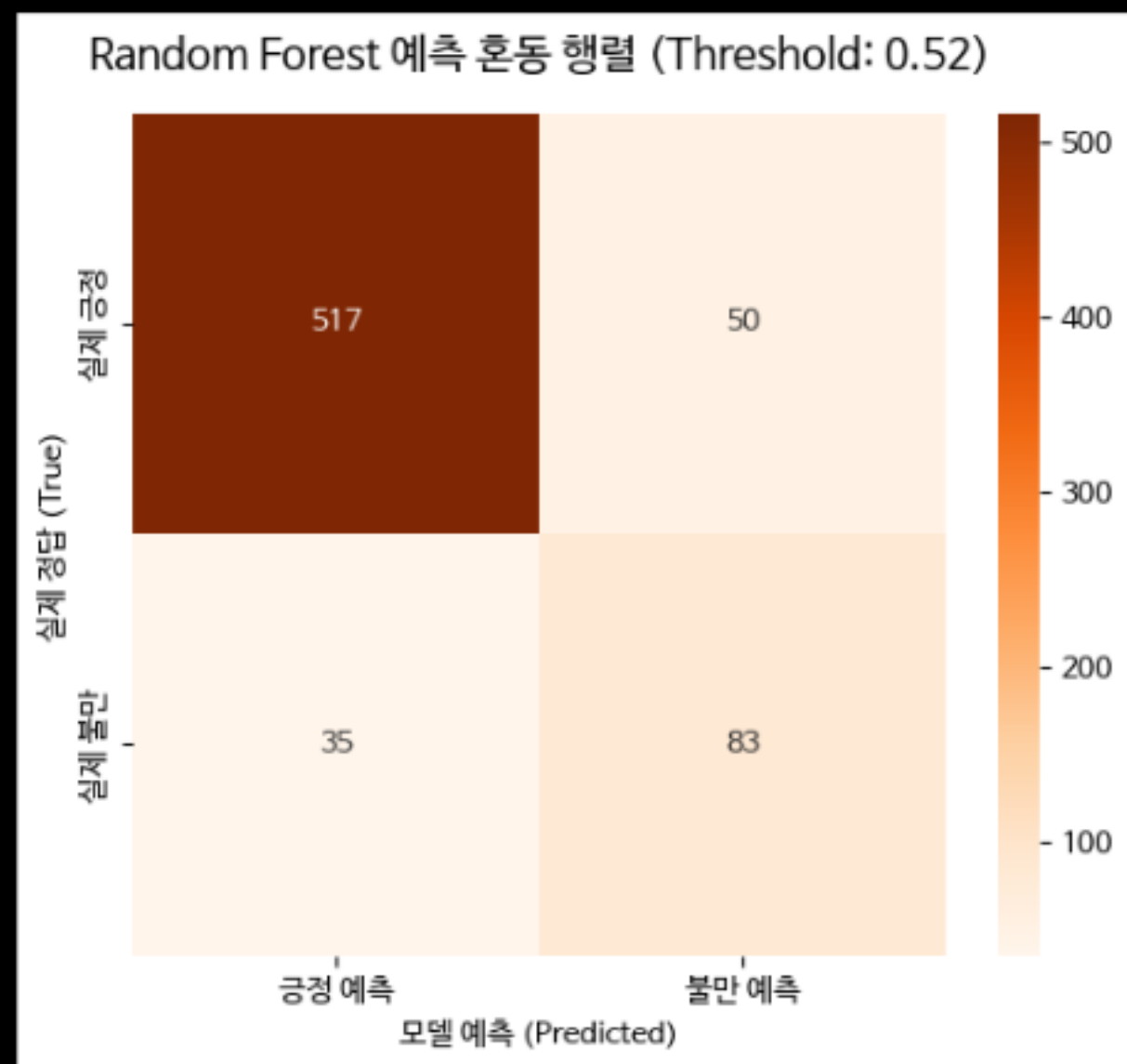
임계값(Threshold) 최적화 탐색 중...

★ 최적의 불만 판정 임계값: 0.58

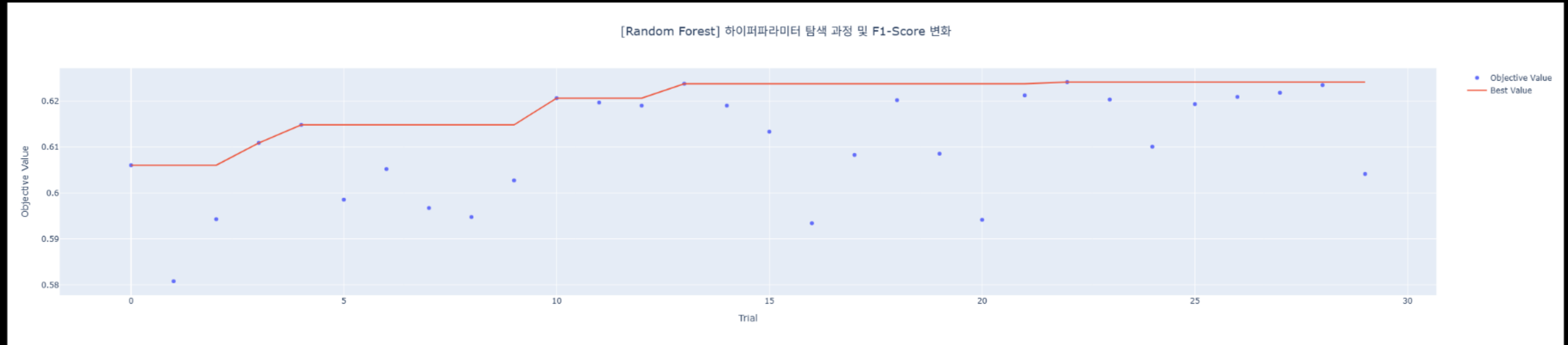
★ 최적 임계값 적용 시 F1-Score: 0.6142

[LightGBM 최적화 모델 평가 결과: Test Data]

	precision	recall	f1-score	support
긍정(0)	0.93	0.88	0.91	567
불만(1)	0.55	0.69	0.61	118
accuracy			0.85	685
macro avg	0.74	0.79	0.76	685
weighted avg	0.87	0.85	0.86	685



• Optuna 하이퍼파라미터 과정을 시각적으로 확인



앙상블 기법 사용 with Soft Voting



· 앙상블(Ensemble)이란?

쉽게 말해 '집단 지성'. 여러 명의 전문가가 회의를 통해 내린 결론이 더 정확하다는 원리.

· 로지스틱과 트리 모델의 예측 결과를 합치는 Voting 기법 사용

· 각 모델이 예측한 확률의 평균을 내어 최종 불만 여부를 결정하면, 단일 모델을 썼을 때보다 F1-Score가 오르는 경우가 많다. 이를 Soft Voting이라고 함.

· Soft Voting: 각 모델이 '이 리뷰가 불만일 확률'을 제출하고, 그 확률들의 평균을 내어 최종 결정.

예) LR: "음, 불만일 확률이 40% 같아. (긍정)

LGBM: "아니야, 파생변수를 보니 불만일 확률이 90%야." (불만)

RF: "나도 보니 불만일 확률이 80%네." (불만)



평균 확률: $(40 + 90 + 80) / 3 = 70\%$
50%를 넘었으므로 최종 불만 판정

• 모델 평가 및 결과 시각화

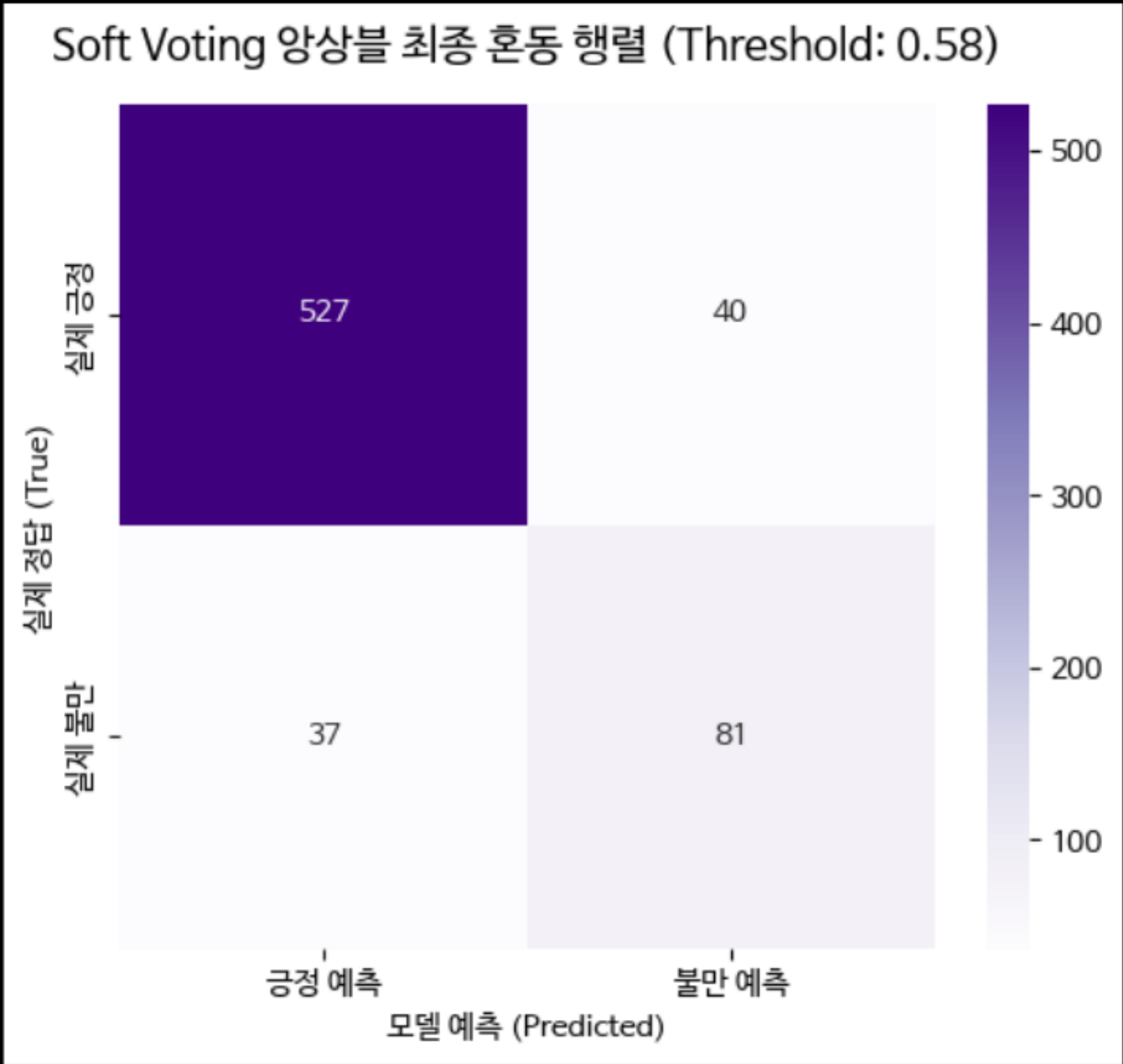
양상블 모델 임계값(Threshold) 최적화 탐색 중...

★ 최적의 양상블 불만 판정 임계값: 0.58

★ 최적 임계값 적용 시 F1-Score: 0.6778

[Soft Voting 양상블 최적화 모델 평가 결과: Test Data]

	precision	recall	f1-score	support
긍정(0)	0.93	0.93	0.93	567
불만(1)	0.67	0.69	0.68	118
accuracy			0.89	685
macro avg	0.80	0.81	0.80	685
weighted avg	0.89	0.89	0.89	685



최적 모델 도출 및 결과 해석



최적 모델 도출

🏆 최종 모델별 평가 결과 (최적 임계값 적용)

	Model_Name	Optimal_Threshold	F1_Score	Recall
0	Soft Voting Ensemble	0.58	0.677824	0.686441
1	Logistic Regression	0.57	0.672065	0.703390
2	Random Forest	0.52	0.661355	0.703390
3	LightGBM	0.58	0.614232	0.694915

🏆 최종 1등 모델: Soft Voting Ensemble
🎯 적용된 최적 임계값(Threshold): 0.58
📊 최고 F1-Score: 0.6778 / Recall: 0.6864

. 앙상블 기법에 대한 시각화

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("앙상블 모델의 방향성(긍정/부정) 통합 중요도를 계산합니다...")

# 1. 텍스트 피쳐와 도메인 피쳐 이름 결합
vocab = tfidf.get_feature_names_out()
all_feature_names = np.concatenate([vocab, domain_features])

# 2. 각 모델의 변수 중요도 및 회귀 계수 추출
lr_coef = model_lr.coef_[0] # 방향성(+,-)을 가진 로지스틱 가중치
lgbm_importance = model_lgbm.feature_importances_
rf_importance = model_rf.feature_importances_

# 3. 중요도 크기 스케일링 (0 ~ 1 사이로 통일하여 공정하게 평균 내기 위해)
scaler = MinMaxScaler()
# LR은 부호를 띄고 절대값(abs) 크기만 먼저 스케일링합니다.
lr_scaled = scaler.fit_transform(np.abs(lr_coef).reshape(-1, 1)).flatten()
lgbm_scaled = scaler.fit_transform(lgbm_importance.reshape(-1, 1)).flatten()
rf_scaled = scaler.fit_transform(rf_importance.reshape(-1, 1)).flatten()

# 4. 통합 앙상블 중요도 '크기' 계산
ensemble_magnitude = (lr_scaled + lgbm_scaled + rf_scaled) / 3.0

# 5. ★ 핵심: 로지스틱 회귀의 부호(Sign)를 빌려와 방향성 부여!
# np.sign()은 양수면 1, 음수면 -1을 반환합니다.
feature_directions = np.sign(lr_coef)
directional_ensemble_importance = ensemble_magnitude * feature_directions

# 6. 데이터프레임 생성
ensemble_df = pd.DataFrame({
    'Feature': all_feature_names,
    'Importance': directional_ensemble_importance
})

# 7. 불만(1, 양수)과 긍정(0, 음수) 핵심 변수 추출
top_negative = ensemble_df.sort_values(by='Importance',
ascending=False).head(15)

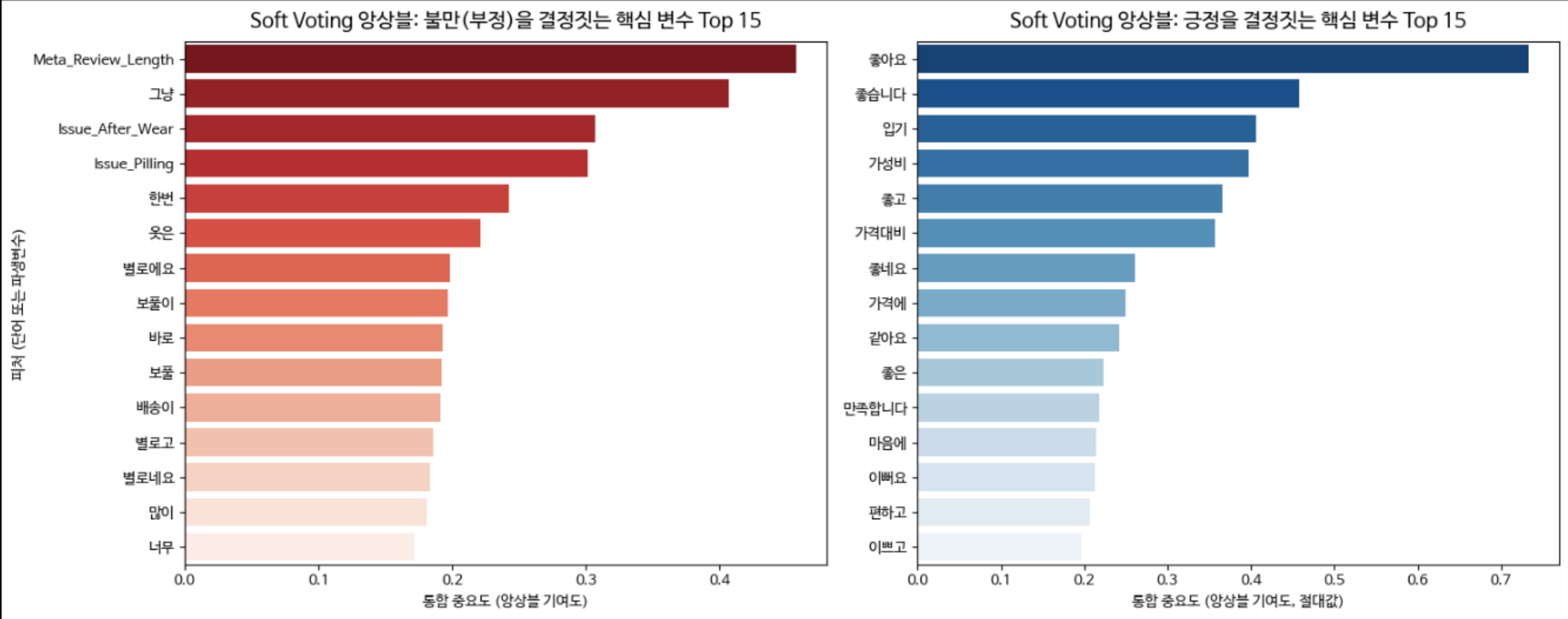
top_positive = ensemble_df.sort_values(by='Importance',
ascending=True).head(15)
top_positive['Abs_Importance'] = np.abs(top_positive['Importance'])
top_positive_sorted = top_positive.sort_values(by='Abs_Importance',
ascending=False)

# 8. 결과 시각화 (좌: 불만, 우: 긍정)
plt.figure(figsize=(15, 6))

plt.subplot(1, 2, 1)
sns.barplot(x='Importance', y='Feature', data=top_negative, hue='Feature',
palette='Reds_r', legend=False)
plt.title('Soft Voting 앙상블: 불만(부정)을 결정짓는 핵심 변수 Top 15',
fontsize=14, pad=10)
plt.xlabel('통합 중요도 (앙상블 기여도)')
plt.ylabel('피쳐 (단어 또는 파생변수)')

plt.subplot(1, 2, 2)
sns.barplot(x='Abs_Importance', y='Feature', data=top_positive_sorted,
hue='Feature', palette='Blues_r', legend=False)
plt.title('Soft Voting 앙상블: 긍정을 결정짓는 핵심 변수 Top 15', fontsize=14,
pad=10)
plt.xlabel('통합 중요도 (앙상블 기여도, 절대값)')
plt.ylabel('')

plt.tight_layout()
plt.show()
```

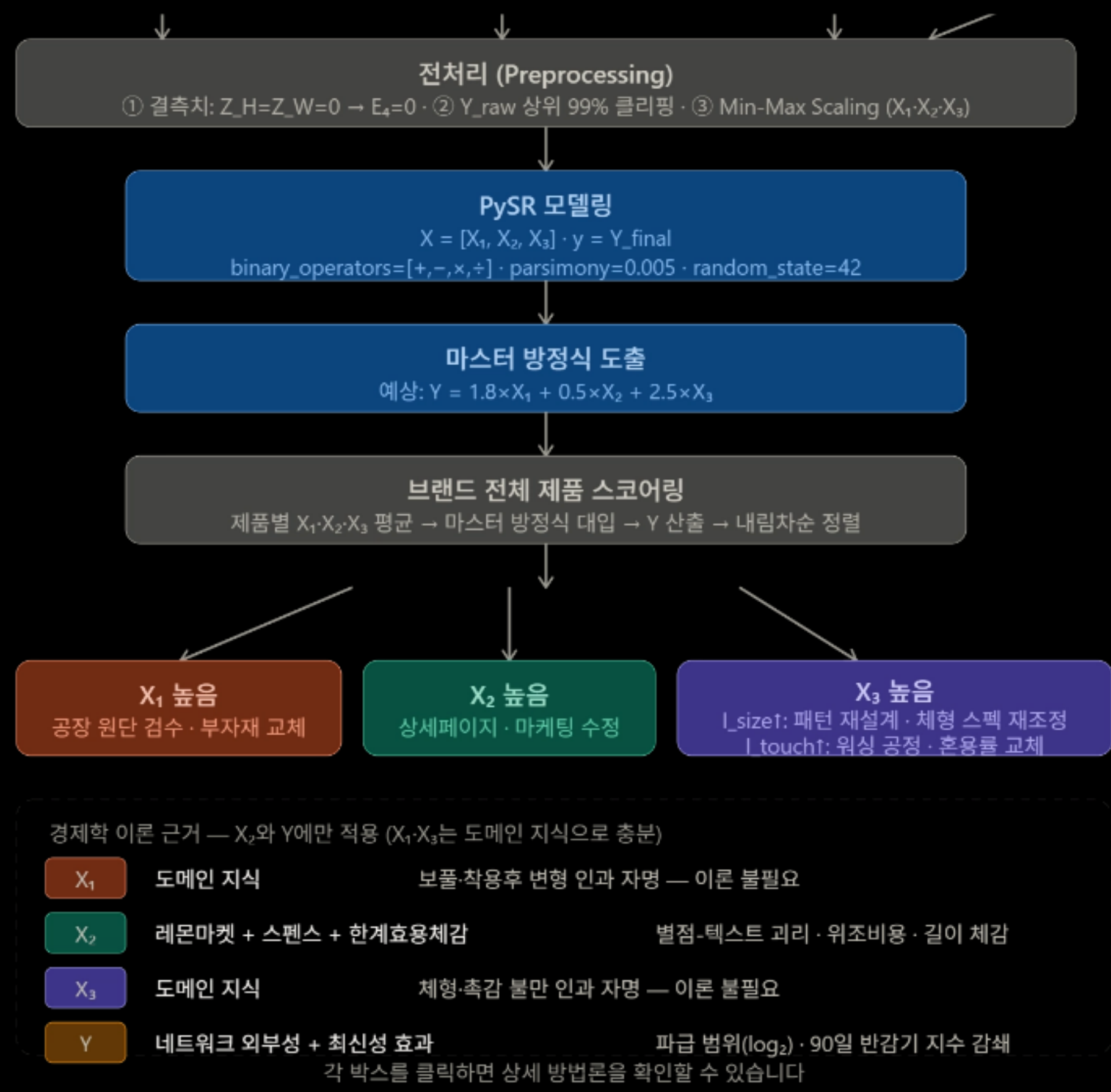


- 불만을 결정짓는 핵심 변수 중 2단계를 진행하며 생상한 파생 변수가 높은 순위를 기록. 이는 데이터를 분석해 만든 파생 변수가 모델 성능에 큰 기여를 했다고 볼 수 있다.

부정 리뷰 기반 R&D 우선순위 자동화 엔진

- 중소 / 신진 디자이너 브랜드 특성상 예산과 인력이 한정되어 있음.
- 따라서 부정 리뷰 데이터 기반의 우선 순위 의사결정을 통해 효과적인 투자 필요.

· 전체 흐름 개요도



우선순위 점수 산출

1.데이터 분석 목적

- 매일 쌓이는 부정 리뷰 — 하지만 ****어떤 제품을 먼저 고쳐야 할지 모른다****
- 예산과 인력은 한정되어 있다
- 지금까지는 직감과 경험에 의존해 왔다
- → ****데이터 기반의 우선순위 의사결정****이 필요하다

2. 독립변수 X 설정

- X1: 객관적 / 물리적 리스크 (제품결함 지수)
- X2: 텍스트 / 감성적 리스크 (리뷰 페널티)
- X3: 주관적 / 개인화 리스크 (사이즈, 촉감 유클리디안 지수)

동별0121 25.07.03

★★★★★1

상품명

워셔블 케이블 반팔 니트 - 16 COLOR

구매옵션

(카라)블랙 · L

체형정보

남성 · 173cm · 67kg

아니 팔 부분 왜 이렇게 짧아요 민소매 티셔츠 인줄 배송은 엄청 늦게 와놓고 색감도 최악이야 귀찮아서 반송은 안하고 후기 남깁니다 진짜 별로야!!!

1) X1 (객관적 결함 지수, 누가 봐도 불량)

$$X_1 = I_{\text{pilling}} + I_{\text{after_wear}} + (I_{\text{pilling}} \times I_{\text{after_wear}})$$

- 보풀만 있는 제품 → 1점
- 세탁 후 줄어들기만 하는 제품 → 1점
- 보풀도 있고 세탁 후 변형까지 있는 제품 → 3점 (폭발적 페널티)

변수	설명
L_pilling	리뷰 텍스트에 '보풀' 관련 불만 여부 (0 또는 1)
L_after_wear	'착용/세탁 후 변형' 불만 여부 (0 또는 1)
L_pilling X L_after_wear	두 결함 동시 발생 시 추가 페널티 (AND 교호작용)

> 왜 단순 합산이 아닌가?
> 두 결함이 동시에 나타나면 단순 불만을 넘어 대량 반품·환불로 직결되기 때문. 이를 수식의 AND 교호작용 항으로 반영.

비즈니스 해석: X1이 높은 제품 → 원사 교체, 공장 검수 기준 강화

2) X2 (별점이 숨기고 있는 불만)

★★★★★ "디자인은 진짜 예뻐서 별점 5점 드려요. 근데 딱 한 번 입었는데 보풀이 미친듯이 생겨서 별로예요."

→ 별점: 5점 / 실제 내용: 심각한 불만

이것이 왜 위험한가? → 레몬마켓 이론

- 별점(숫자)만 믿고 샀다가 원단이 별로네 → 해당 브랜드 전체를 불신
- 텍스트 리뷰 속 "겉과 속이 다른" 신호를 잡아내야 한다

레몬마켓



- 판매자와 소비자 사이 **정보의 비대칭성**이 큼
- 소비자들이 품질이 안 좋은 상품을 구매할 것을 우려해 낮은 가격에 재화를 구매하고자 함
- 저품질의 재화와 서비스가 판매됨

결과
: 불량품(가치 없는 재화)만 많아져 소비자가 외면

• X2 수치화

스펜스 신호이론 적용

- 정보가 비대칭적인 상황에서, 우수한 정보를 가진 쪽이 자신의 가치를 입증하기 위해 를 보내어 상대방의 불신을 해소하는 과정을 설명하는 경제학 이론
- 긴 리뷰 = 작성자가 시간을 들였다 = 진짜 경험에서 나온 불만일 가능성 높음

$$X_2 = (N_{\text{but}} + N_{\text{neg_adverb}}) \times (1 - \exp(-L_{\text{review}} / \mu_L))$$

변수	설명
N_but	역접 접속사 수 (근데, 하지만 등)
N_neg_adverb	부정 강조 부사 수 (너무, 진짜, 완전 등)
L_review	리뷰 텍스트 길이 (문자 수)
μ_L	브랜드 내 전체 리뷰 길이의 평균

- 왜 두 요소를 곱하는가?
- 내용이 강해도 리뷰가 짧으면 → 신뢰도 낮음
 - 리뷰가 길어도 내용이 없으면 → 강도 낮음
 - 둘 다 충족될 때만 높은 점수 → AND 논리

비즈니스 해석: X_2 가 높은 제품 → 상세페이지 수정, 마케팅 문구 조정

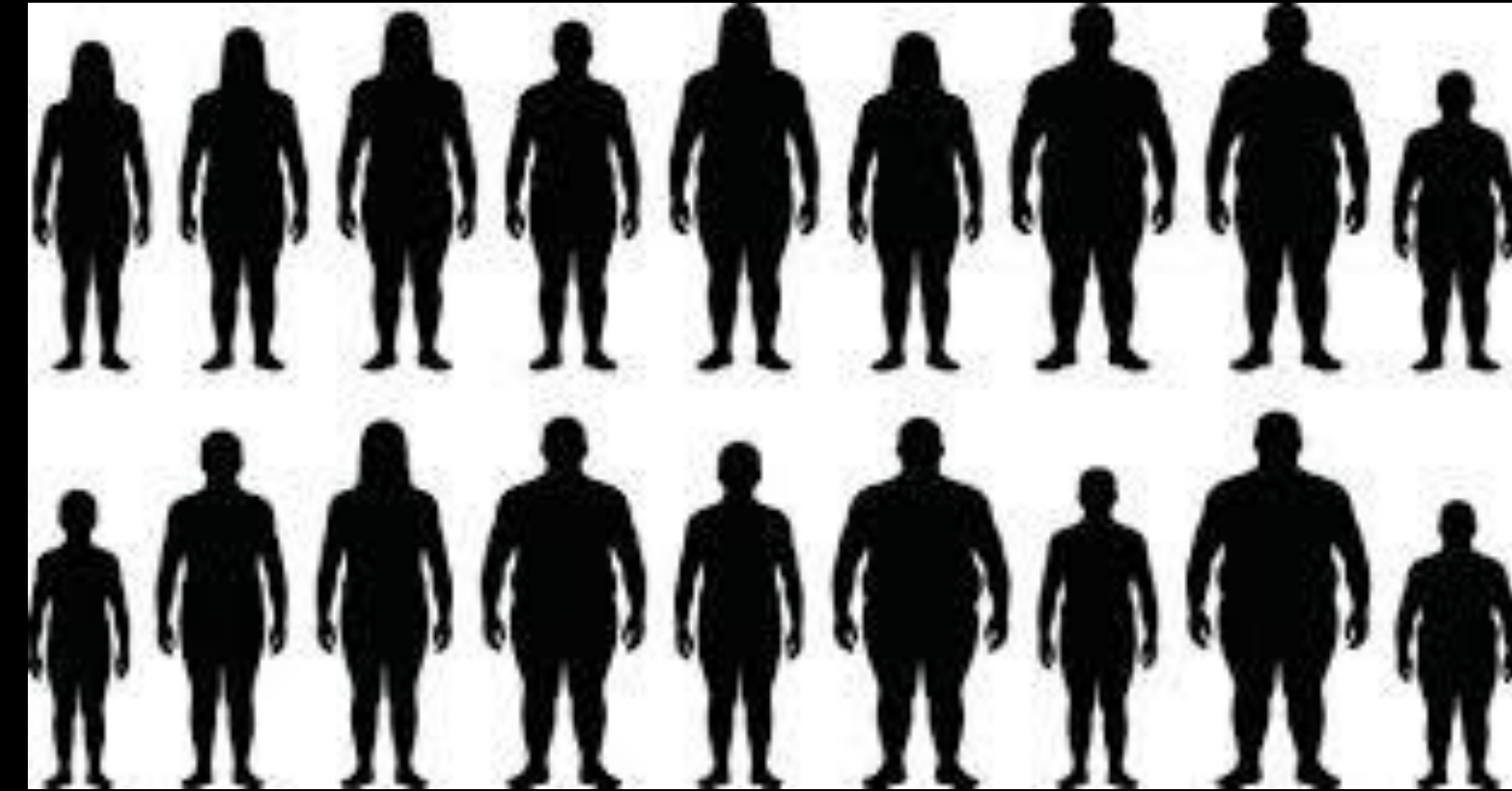
3) X3 (주관적 불만)

주관적 불만이란?

- "사이즈가 안 맞아요" → 내 체형 문제일 수도, 제품 설계 문제일 수도
- "촉감이 까슬거려요" → 내 피부가 민감한 걸 수도, 원단 문제일 수도

그냥 무시하면 안 되는 이유

- > 키 190cm에 몸무게 60kg인 고객 10명이 모두 "팔이 짧아요"라고 한다면?
→ 개인 취향이 아니라 브랜드의 **설계 자체 오류**



핵심 아이디어

- 표준 체형에서 크게 벗어난 고객이 사이즈 불만을 낼 때 → 브랜드 디렉터에게 "이 옷의 패턴에 문제가 있다"는 강력한 시그널
- 단, 같은 특이 체형 고객이 촉감 불만을 낼 때 → 체형과 무관한 문제이므로 가중치 없이 처리

• X3 수치화

$$X_3 = \sqrt{((I_size \times (1 + E_4))^2 + I_touch^2)}$$
$$*E_4 = \sqrt{(1/(1-r^2)) \times (Z_H^2 + Z_W^2 - 2r \cdot Z_H \cdot Z_W))}$$

- 마할라노비스 거리: 키와 몸무게의 상관관계를 함께 고려한 "진짜 이탈도"
- 키와 몸무게를 동시에 봤을 때 얼마나 표준에서 벗어났는지를 측정

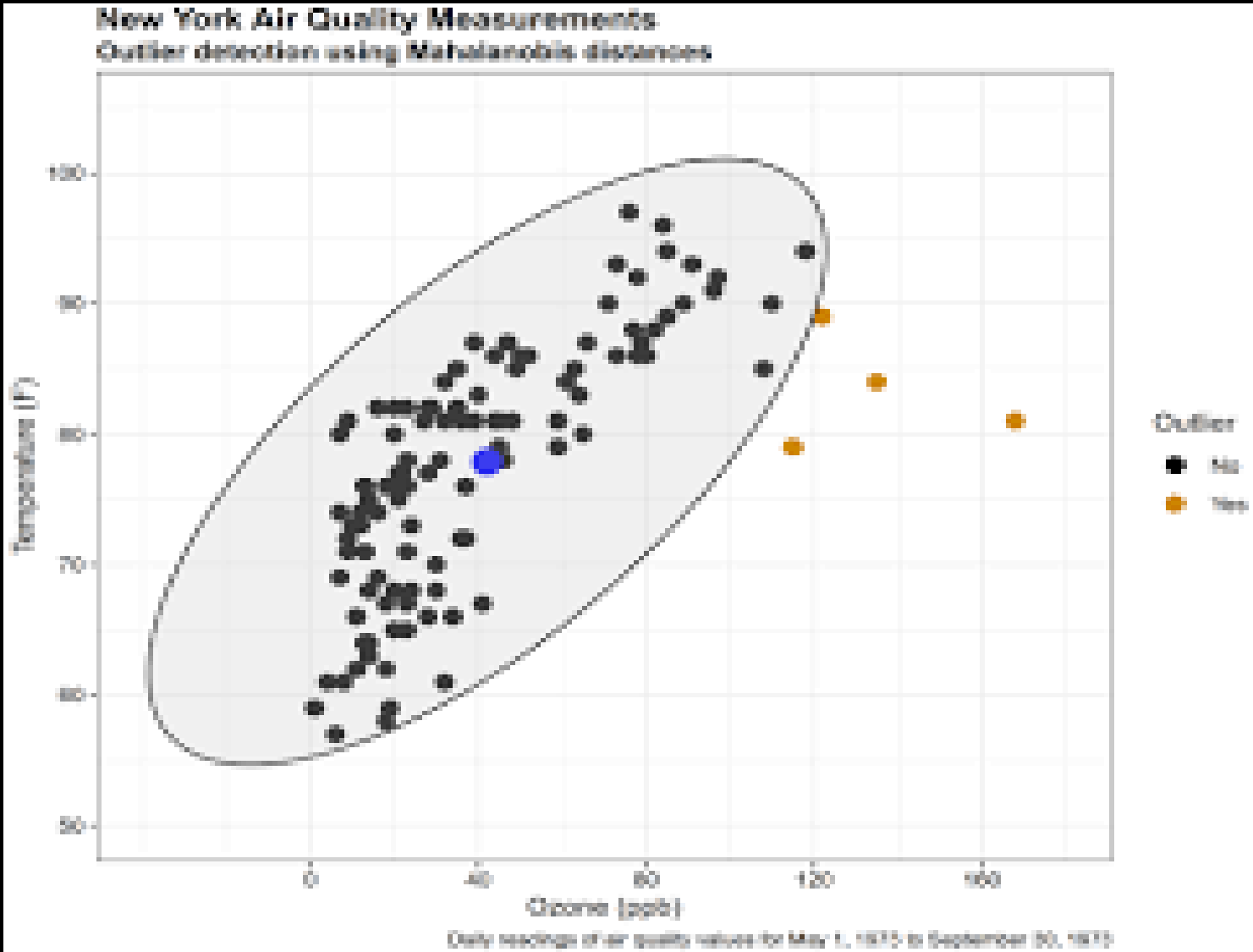
X3 최종 구조

- 특이 체형일수록 사이즈 페널티 폭발
- 촉감 불만 → 체형 가중치 없이 그대로 반영
- 두 차원(사이즈, 촉감)을 유클리디안 거리로 통합

비즈니스 해석:

- X3 사이즈가 높은 제품 → 패턴 재설계, 체형별 스펙 재조정
- X3 촉감이 높은 제품 → 워싱 공정 추가, 혼용률 교체

변수	설명
I_size	역접 접속사 수 (근데, 하지만 등)
I_touch	부정 강조 부사 수 (너무, 진짜, 완전 등)
Z_H	리뷰 텍스트 길이 (문자 수)
Z_W	브랜드 내 전체 리뷰 길이의 평균
r	키와 몸무게의 피어슨 상관계수
E4	마할라노비스 거리-체형 이탈도



3. 타겟변수 Y 설정

1) 타겟변수 Y

기본 위험도 * 시간 감가상각 = ' 현재 브랜드가 가지고 있는 문제수준'

2) 타겟변수 도출 과정

① 1단계 기본 위험도 Y_raw

```
Y_raw = (5 - rating) × [1 + log2(1 + helpful) + (is_photo × 0.5)]
```

변수	설명
rating	별점 (1~5점)
helpful	다른 고객이 누른 '도움이 돼요' 수
is_photo	포토 리뷰 여부 (0 또는 1)

구분	설명
(5 - rating)	별점이 낮을수록 기본 위험도 상
log ₂ (1 + helpful)	파급력, 한계효용체감 적용
is_photo × 0.5	사진 첨부 시 파급력 추가 보너스

③ 2단계 시간 감가상각 E₆ (오래된 리뷰일 수록 위험도 ↓)

```
E6 = exp(-ln(2)/90 × Δt)
```

변수	설명
Δt	리뷰 작성일로부터 현재까지 경과 일수

④ 최종타겟

```
Y_final = Y_raw × E6
```

4. 전처리

작업	내용	이유
결측치 처리	키/몸무게 미입력 → 평균값(표준 체형)으로 대체	데이터 없는 고객을 보수적으로 처리
이상치 통제	Y 점수 상위 1%는 상한선 적용	공감 수 조작 등 비정상 데이터 차단
정규화	X_1, X_2, X_3 를 모두 0~1 사이로 변환	세 변수를 공정하게 비교하기 위해

5. PySR 모델링

PySR: 수식을 스스로 찾아내는 AI

- X_1, X_2, X_3 를 조합해서 Y 를 가장 잘 설명하는 수식을 자동으로 탐색하는 도구
- 유전자 알고리즘 기반: 수많은 수식 조합을 시도하며 가장 단순하고 정확한 것을 선택

	일반 회귀분석	PySR
수식 구조	사람이 미리 가정 ($Y = aX + b$)	데이터에서 스스로 발견
결과	계수만 도출	수식 구조 자체를 도출
해석 가능성	높음	높음 (사칙연산만 허용 시)

핵심 설정 2가지

- 사칙연산만 허용 → 수식을 사람이 이해할 수 있게 유지
- 복잡한 수식에 페널티 → 단순하고 해석 가능한 수식 선택

7. Hall of Fame: Final 방정식 후보 탐색

Complexity	Loss	Score	Equation
1	9.624e-02	0.000e+00	$y = 0.069146$
5	9.616e-02	2.028e-04	$y = (X3 * -0.11145) - -0.070564$
7	9.616e-02	8.941e-08	$y = ((X3 * -0.038122) / 0.34064) + 0.070565$
9	9.615e-02	9.368e-05	$y = ((X3 + -0.6369) - (X1 * 0.26459)) * -0.1087$
11	9.607e-02	3.689e-04	$y = ((X2 - (X2 * X2)) * (X2 - 0.34957)) + 0.071116$
13	9.606e-02	7.672e-05	$y = ((X2 - (X2 * X2)) * (X2 + (-0.32974 - X3))) - -0.07101$
15	9.599e-02	3.821e-04	$y = ((X2 - (X2 * X2)) * (X2 - 0.34643)) + (0.072564 - (X3 ... * 0.11903))$

Index	복잡도	오차(Loss)	특징
0~3	낮음 (1~5)	높음	수식이 단순 → X1 또는 X2만 포함, 정확도 낮음
4~7	중간 (6~15)	중간	일부 변수 조합, 해석 가능하나 불완전
8~10	높음 (16~25)	낮음	X1·X2·X3 모두 등장 시작
11+	매우 높음 (26+)	매우 낮음	정확하지만 수식이 너무 복잡해 해석 불가

왜 Index 10 (복잡도, 오차율 0.096)을 채택했는가?

- X1, X2, X3 세 변수가 모두 포함된 유일한 후보
→ 세 리스크 축을 모두 반영해야 한다는 설계 의도와 일치
- 복잡도 25: 수식이 길지만 사람이 항목별로 읽고 해석 가능한 수준
- 오차율 0.096: 실용적 정확도 확보
- Index 11 이상: 정확도는 더 높지만 수식 구조가 블랙박스화

단순함 vs 정확도의 트레이드오프에서,
→ "현업에서 설명 가능한 수식" 을 최우선 기준으로 선택.

```
=====
🏆 PySR01 도출한 브랜드 제품 문제 우선순위 방정식 결과
=====
PySRRegressor.equations_ = [
    pick      score      equation
0          0.000000e+00      0.06914611
1          2.123947e-04      0.008969816 / (X3 + 0.12659584)
2          3.738850e-05      ((X3 + -0.63142824) * -0.10967297) - (X1 * -0.1...
3          3.687237e-04      ((X2 - (X2 * X2)) * (X2 - 0.34956622)) + 0.071...
4          7.668597e-05      ((X2 - (X2 * X2)) * (X2 + (-0.3297398 - X3))) ...
5 >>>> 3.822520e-04      ((X2 - 0.34642968) * (X2 - (X2 * X2))) + (0.07...
6          7.267187e-05      (((X2 - (X2 * X2)) * ((X2 - 0.34141758) / 0.75...
7          2.396523e-07      (0.03514854 - (X3 * 0.12176813)) + (((X2 - 0.3...
8          2.732112e-05      (((X2 - 0.34332818) * (X2 - (X2 * (X2 + 0.056...
9          5.298972e-05      ((X2 - 0.34636533) * ((X2 - (X2 * X2)) / 0.747...
10         2.063430e-06      (((0.12095804 - X3) * ((X1 * 0.2192255) + 0.11...
11         2.300053e-05      (((X1 * 0.19996099) + 0.083100565) * (0.10377...
```

8. 마스터 방정식 확정

채택된 방정식 (Index 10 | 복잡도 25 | 오차율 0.096):

$$Y = (((0.121 - X_3) \times ((X_1 \times 0.219) + 0.115)) - 0.378) + (((X_2 - 0.344) \times (X_2 - X_2^2)) / 0.749 + 0.436)$$

항	복잡도	오차(Loss)
항 A	$(0.121 - X_3) \times (X_1 \times 0.219 + 0.115)$	X_3 (사이즈·촉감 리스크)가 클수록 Y를 낮추는 방향으로 작용. 단, X_1 (결함)이 클수록 이 감소폭이 커짐 → X_1 과 X_3 의 상호작용
항 B	$(X_2 - 0.344) \times (X_2 - X_2^2) / 0.749$	X_2 (텍스트 리스크)가 특정 구간(0.344~1.0)에서만 Y를 강하게 올림 → X_2^2 이 들어가 비선형 효과

핵심 해석 3가지

- ① X_3 는 직접적으로 Y를 높이지 않는다

→ X_3 가 크면 항 A 전체가 음수가 되어 Y를 낮추는 방향

→ 단, X_1 이 동시에 높을 때 이 효과가 반전되어 리스크가 폭발

→ "보풀+사이즈 동시 불만"이 가장 위험한 조합임을 수식이 직접 말해줌
- ② X_2 는 특정 구간에서만 폭발적으로 작용한다

→ X_2 가 0에 가깝거나(짧고 단순한 부정 리뷰) 1에 가까우면 효과 약화

→ X_2 가 0.344~0.7 구간(적당히 길고 역접 접속사 있는 리뷰)일 때 Y에 가장 크게 기여

→ "길지는 않지만 내용 있는 부정 리뷰"가 실제로 가장 위험
- ③ 이 수식은 단순 가중합($aX_1 + bX_2 + cX_3$)이 아니다

→ PySR이 변수 간 상호작용과 비선형성을 데이터에서 직접 발견

→ 사람이 임의로 설계한 수식이 아닌, 실제 데이터가 스스로 도출한 패턴

9. 마스터 방정식 적용 결과: 항별 기여도 분석

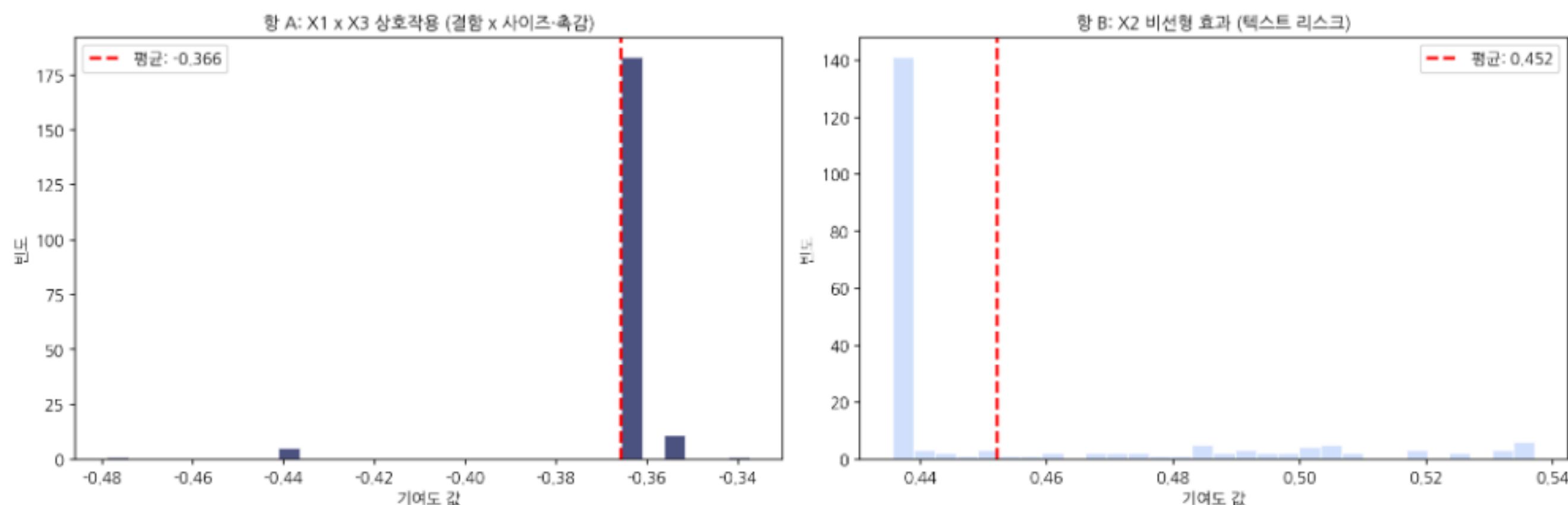
· 방정식 구조 복습

- 항 A = X_1 (객관적결함)과 X_3 (사이즈·촉감, 주관적결함)의 상호작용 → 두 리스크가 동시에 높을 때 폭발
- 항 B = X_2 (텍스트 리스크)의 비선형 효과 → 특정 구간에서만 강하게 작용

· 실제 데이터 적용 결과 해석

- 항 A 분포: 대부분의 리뷰에서 값이 낮게 유지됨
→ X_1 (보풀·변형)이 실제로 발생한 리뷰가 소수이기 때문
→ 소수지만 발생 시 리스크가 폭발적으로 커지는 구조 확인
- 항 B 분포: 리뷰마다 편차가 더 크게 나타남
→ X_2 (텍스트 신호)는 거의 모든 리뷰에서 작동하는 상시 변수
→ 일상적으로 쌓이는 위험 신호를 잡아내는 역할

마스터 방정식 항별 기여도 분포



항 A = 저빈도·고강도 리스크 탐지기
항 B = 고빈도·일상적 리스크 모니터링

10. X2 비선형 효과: 어느 리뷰가 가장 위험한가?

X2 구간	리뷰 특성	위험도 기여
0 ~ 0.34	짧고 단순한 부정 표현	낮음
0.34 ~ 0.70	적당한 길이 + 역접 접속사·부정 부사 존재	최고
0.70 ~ 1.0	매우 긴 리뷰 (길이만 길고 내용 희석)	다시 감소

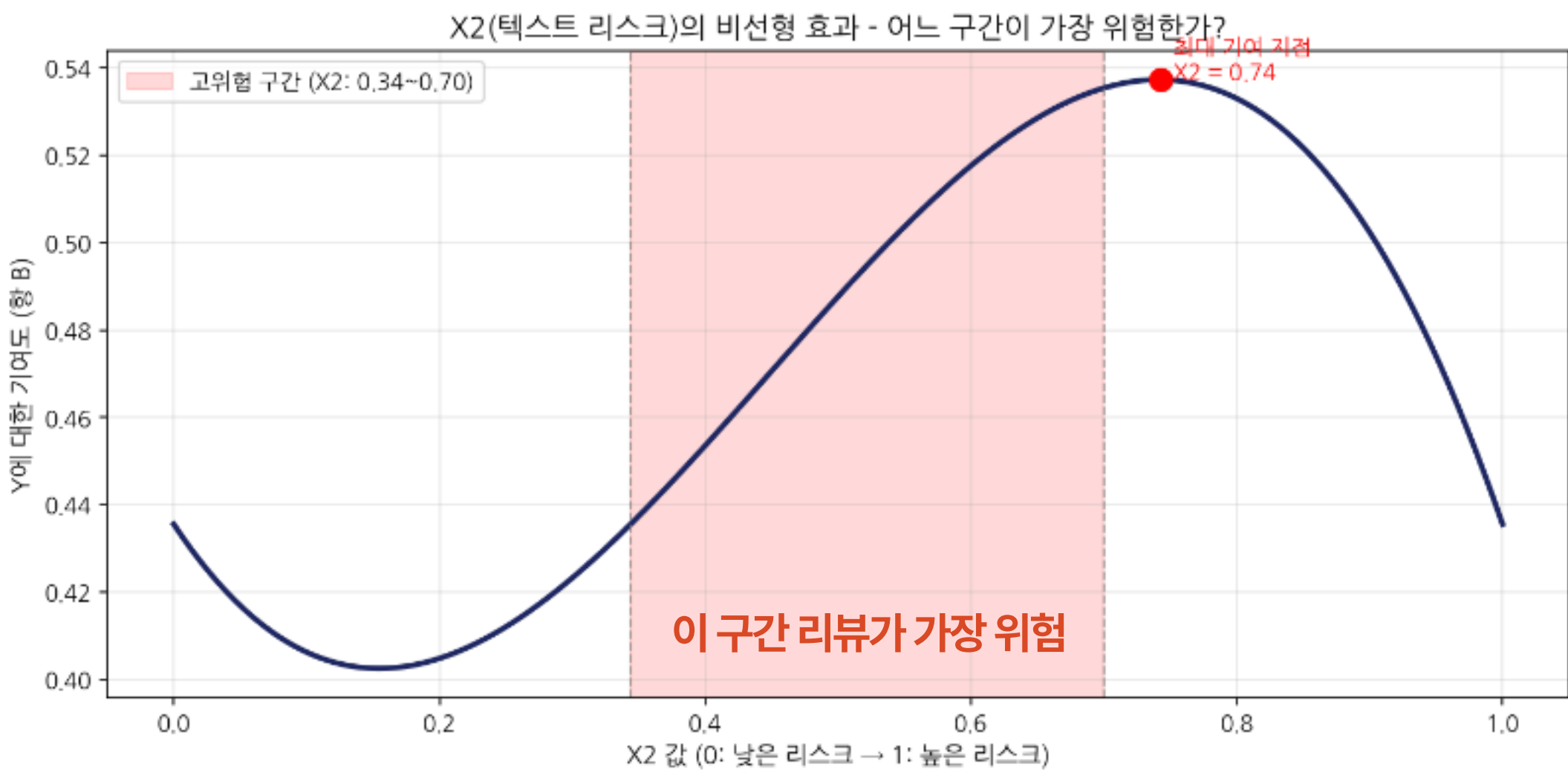
· 실전 해석

"디자인은 진짜 예뻐서 별점 5점 드려요. 근데 딱 한 번 입었는데 보풀이 미친듯이 생겨서 별로예요."

- 이 리뷰는 X2가 고위험 구간(0.34~0.70)에 정확히 위치
- 별점은 5점이지만 Y(위험도) 기여가 가장 큰 유형

왜 너무 긴 리뷰는 오히려 위험도가 낮아지는가?

- 길이가 길수록 신호의 강도는 희석됨
- X_2^2 항이 포화를 만들어 무한정 증가를 차단
- 한계효용체감 법칙이 수식 안에 자동으로 반영된 결과



11. 위험도 분포 및 상위 위험 리뷰

전체 Y 분포에서 읽히는 것

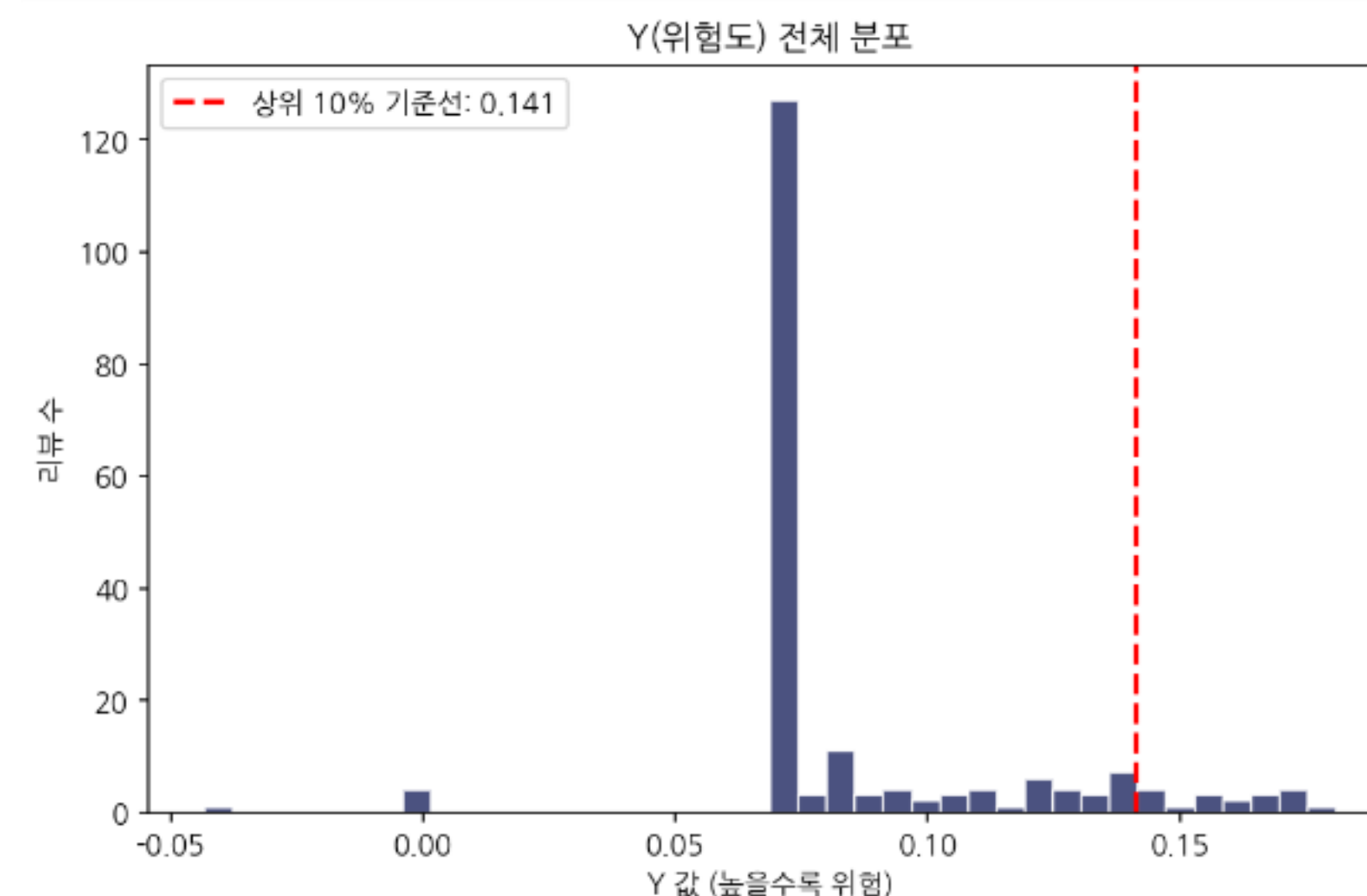
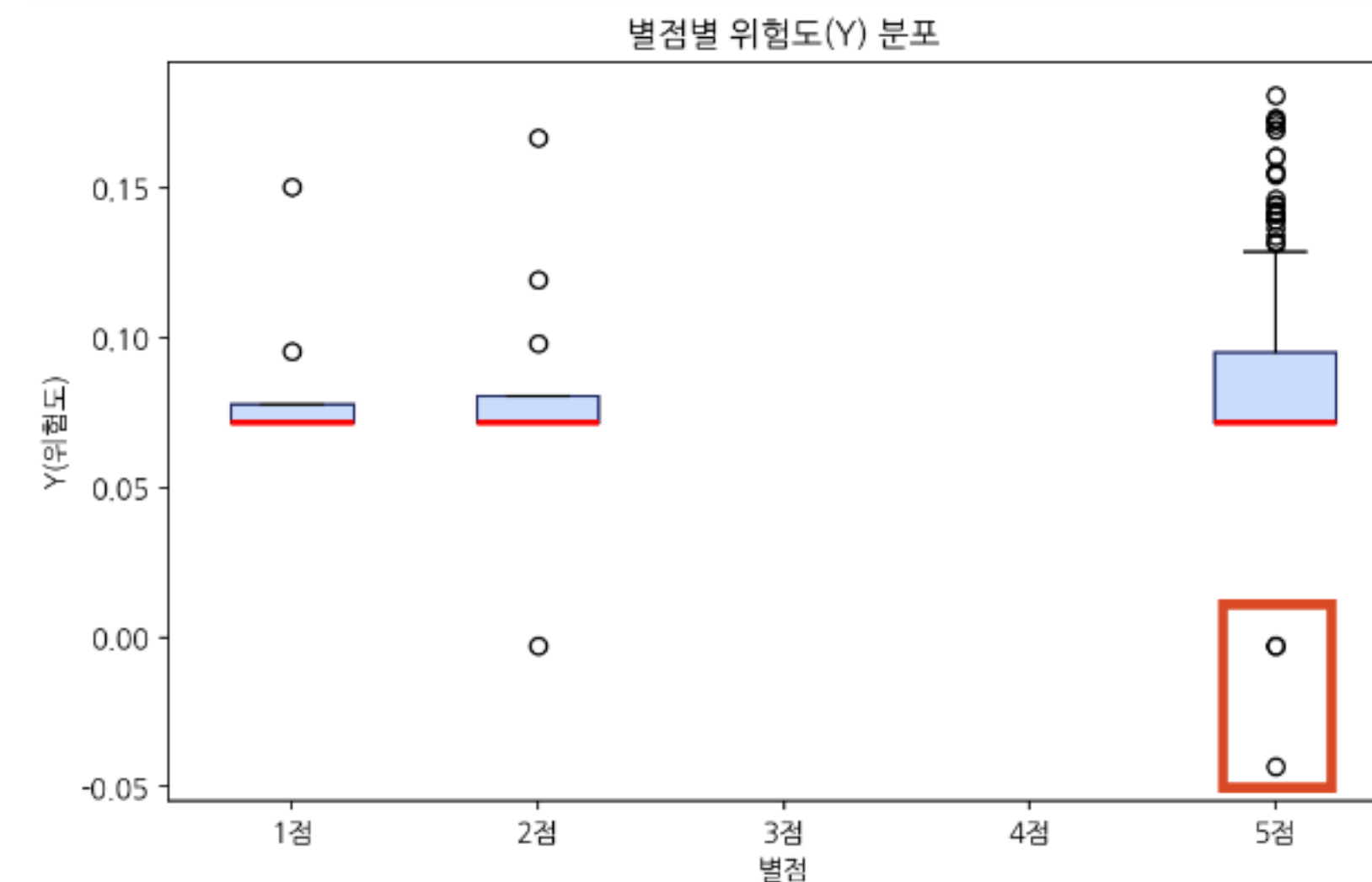
- 대부분의 리뷰는 Y가 낮은 구간에 몰림 → 수아레 니트는 전반적으로 만족도 높은 제품
- 하지만 상위 10%의 고위험 리뷰가 존재 → 이 리뷰들이 실제 브랜드 리스크를 만드는 주범

별점별 Y 분포 해석

- 1~2점 리뷰: Y 중앙값이 높고 이상치도 크게 존재
- 3~5점 리뷰에서도 Y가 높은 이상치 존재 → 이것이 레몬마켓 리뷰
→ 별점만 보서는 절대 잡을 수 없는 위험 신호

상위 위험 리뷰 Top 10

- Step 5 텍스트 출력에서 실제 리뷰 내용 확인
- 공통점: X1 또는 X2가 높고, 최근 날짜인 경우 Y_master가 특히 높음



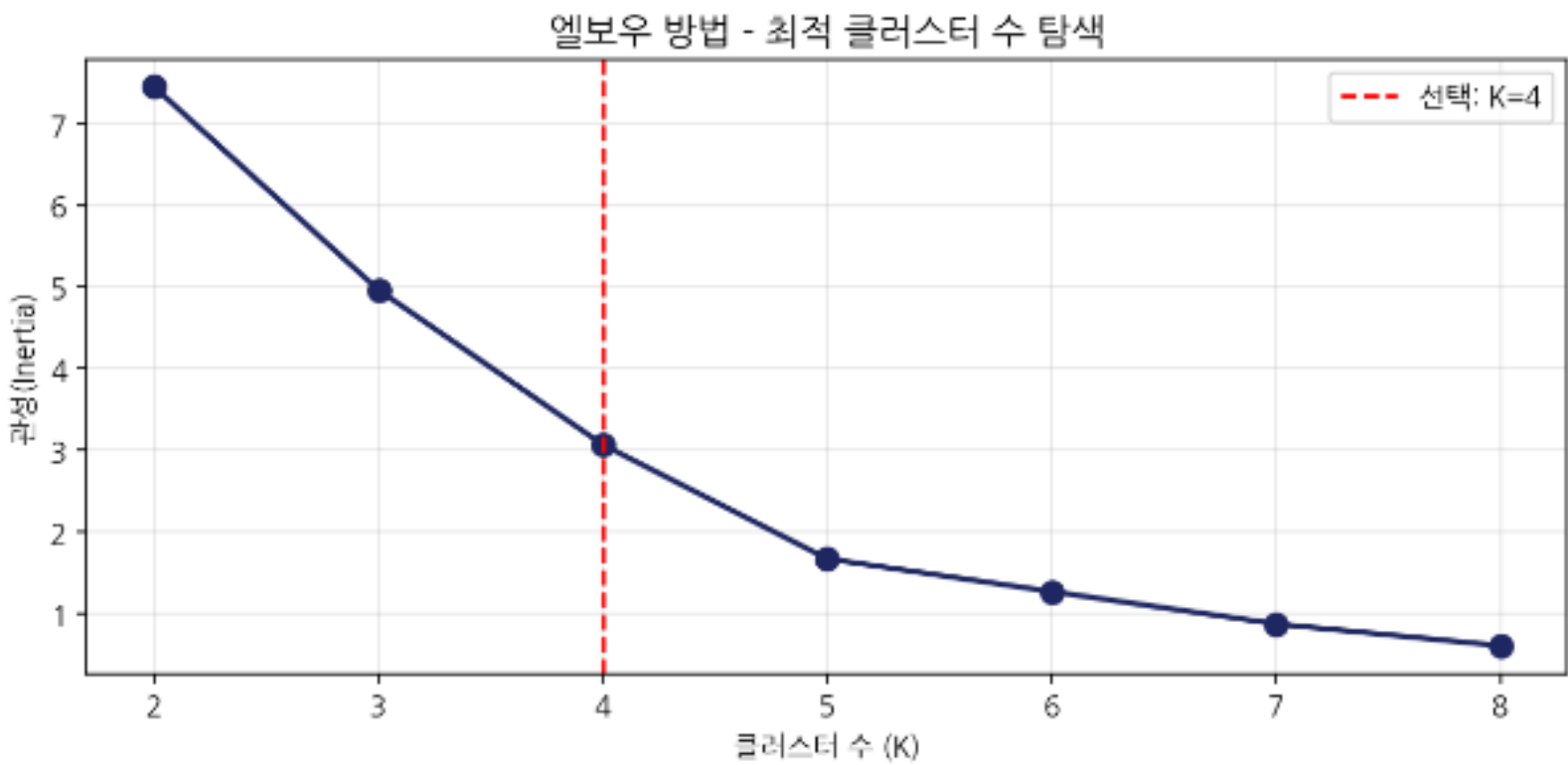
12. K-Means 클러스터링: 리뷰를 4개 유형으로 분류

왜 클러스터링을 하는가?

- Y 점수 하나로 "얼마나 위험한가"만 알 수 있음
- 클러스터링을 통해 "어떤 종류의 위험인가" 를 파악할 수 있음 → 원인별 맞춤 액션 설계 가능

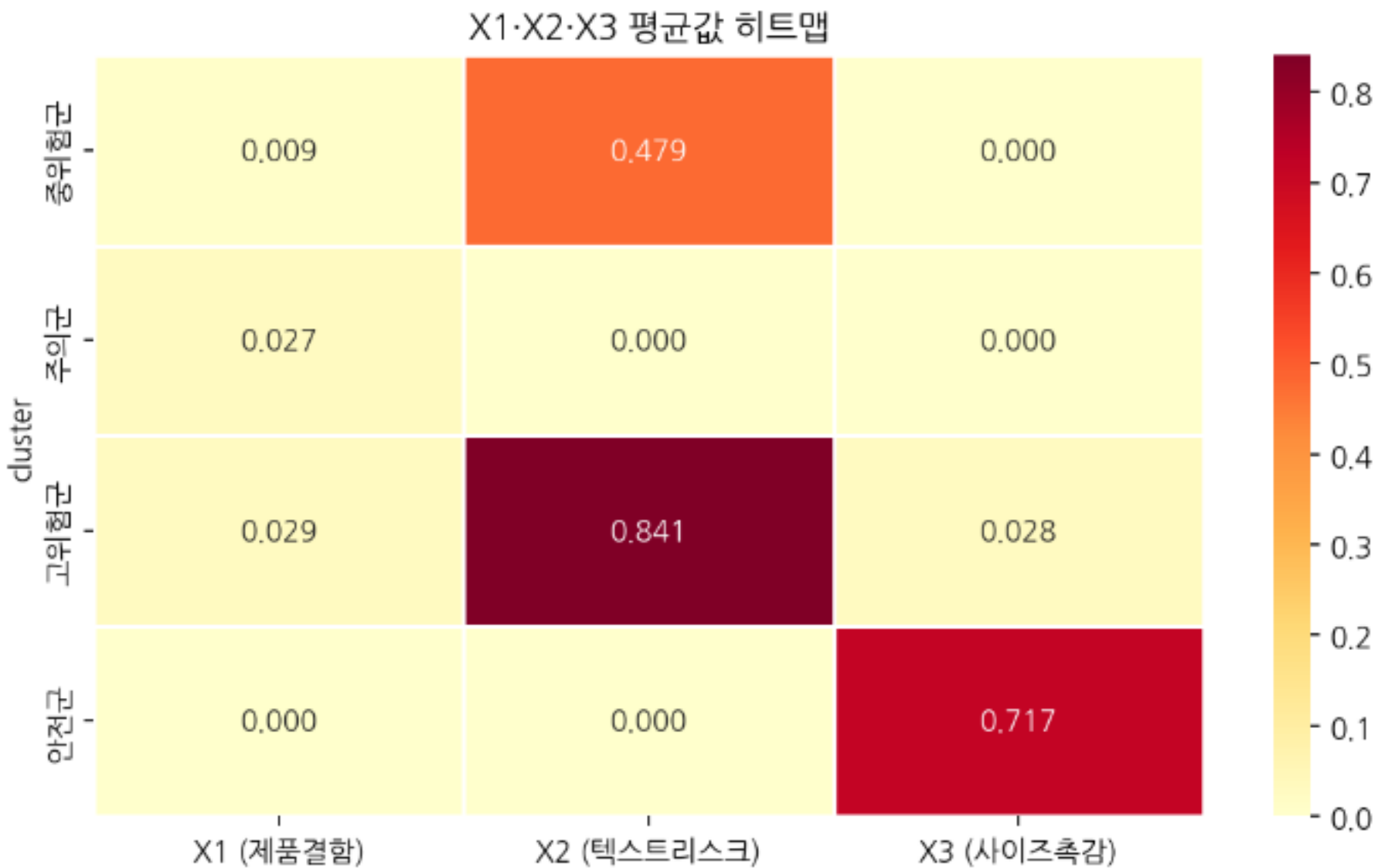
K = 4 선택 근거

- 엘보우 방법으로 K=2~8 탐색 → K=4에서 관성 감소 기울기가 완만해짐
- 비즈니스적으로도 4가지 문제 유형(공정불량/정보오류/도안오류/타겟층오류)과 자연스럽게 매핑



4개 클러스터 특성

클러스터	주요 특징	핵심 원인
고위험군	X1·X2·X3 복수 리스크 높음	복합 문제
중위험군	X2 주로 높음	정보 비대칭
주의군	X3 주로 높음	패턴·원단 문제
안전군	전체 낮음	만족 고객



13. 클러스터 분포 시각화 (3D)

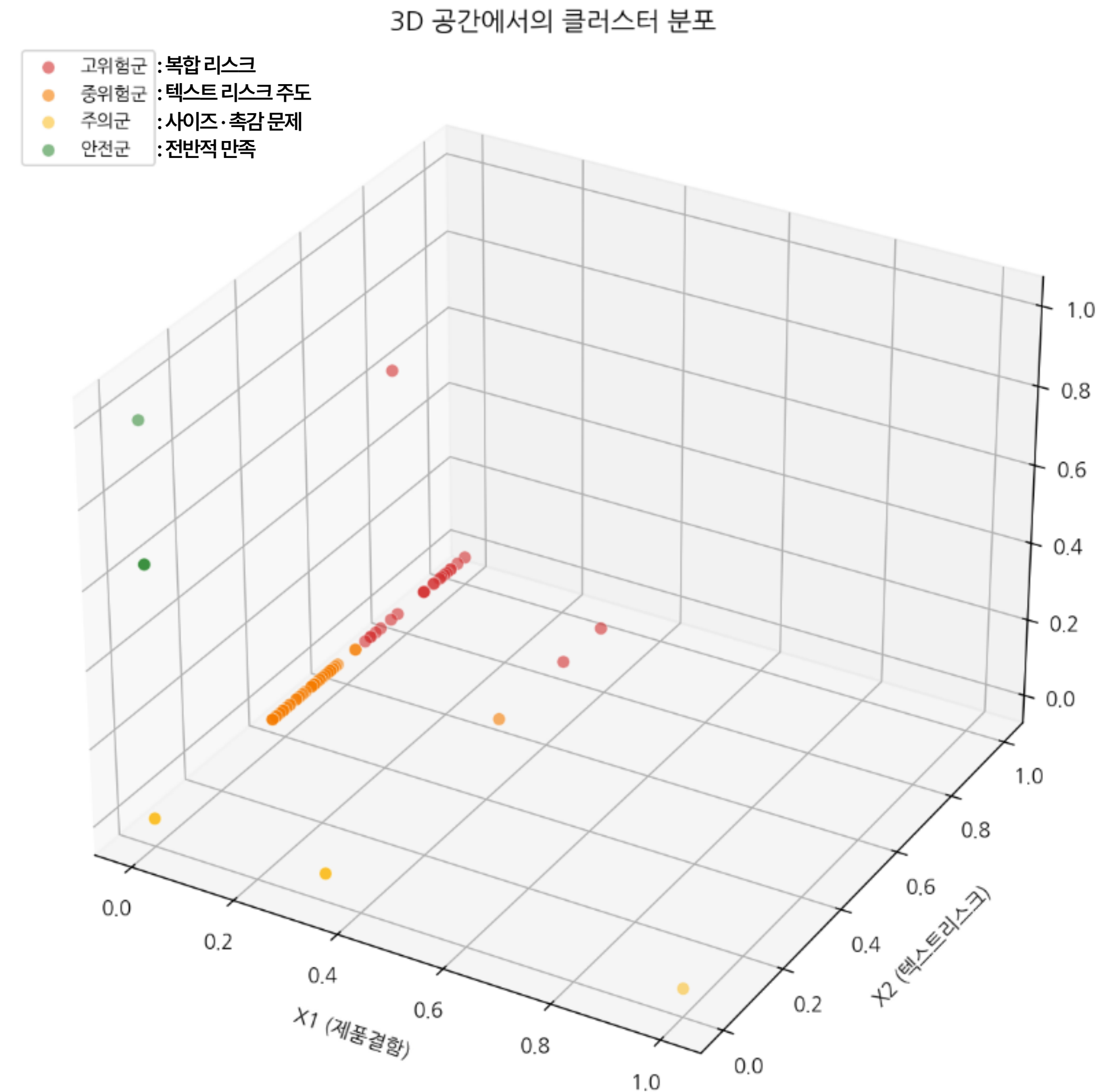
3D 산점도가 보여주는 것

- X축 (X₁, 제품결함), Y축 (X₂, 텍스트리스크), Z축 (X₃, 사이즈·촉감)
- 각 점 = 리뷰 한 건
- 색상 = 클러스터 (고위험군·중위험군·주의군·안전군)

읽는 방법

- 빨간 점(고위험군): 3D 공간 상단·외곽에 위치 → 여러 리스크 축에서 동시에 높음
- 초록 점(안전군): 원점 근처에 밀집 → 모든 리스크 낮음
- 중간 색상들: 특정 축 방향으로만 치우쳐 있음 → 한 가지 리스크 원인

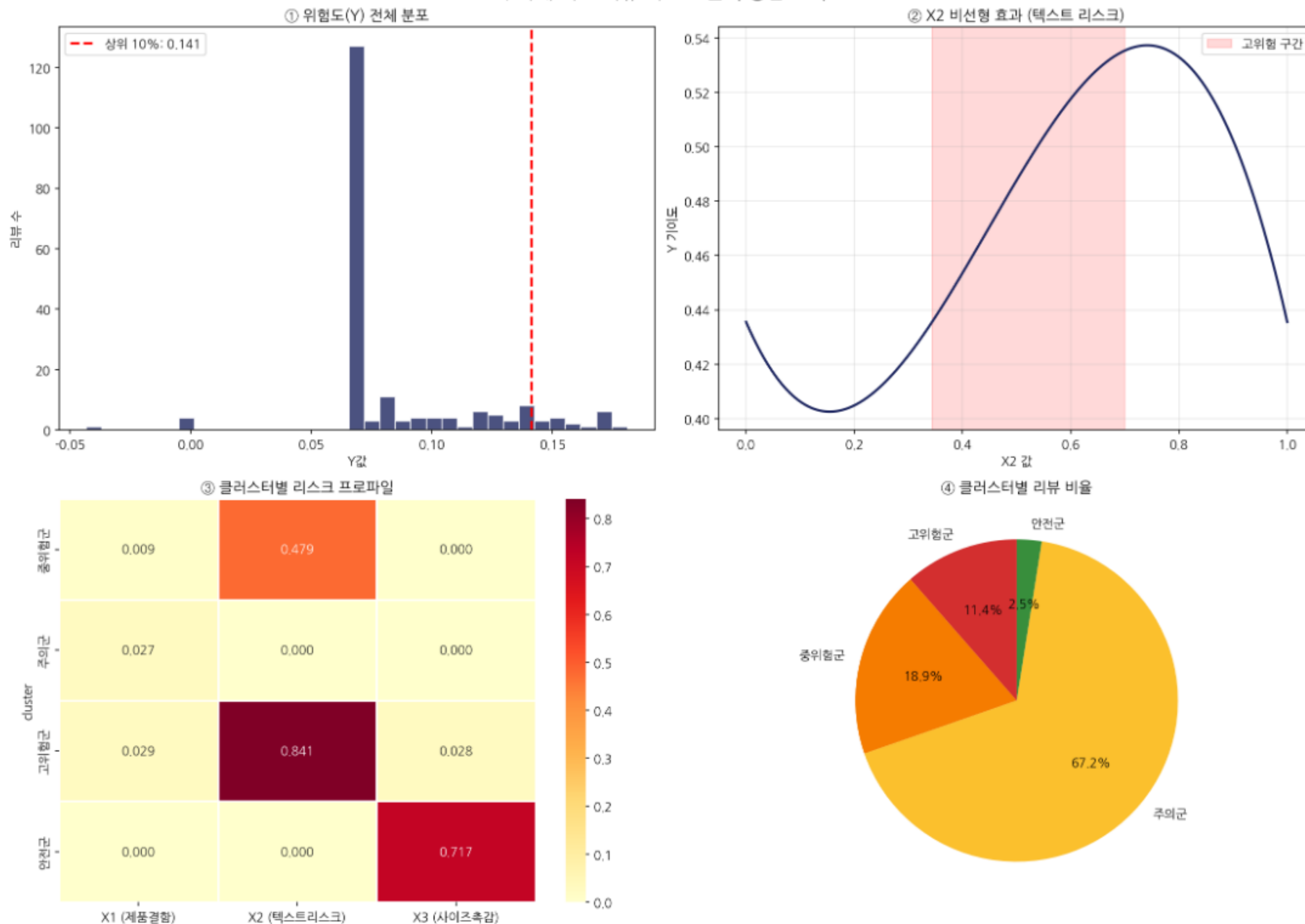
> 3D 시각화로 클러스터가 리스크 공간에서 실제로 분리되어 있음을 확인.



14. 종합 요약

- ① 위험도(Y) 전체 분포
→ 대부분 안전, 소수의 고위험 리뷰 존재
- ② X2 비선형 효과
→ 중간 길이 부정 리뷰가 가장 위험
- ③ 클러스터별 리스크 프로파일
→ 4가지 불만 유형 확인
- ④ 클러스터 비율
→ 안전군이 다수, 고위험군이 소수지만 집중 관리 필요

수아레 니트 리뷰 리스크 분석 종합 요약



프로젝트 한계 및 개선 방향

현재 한계

- 수아레 브랜드의 니트류 제품에만 적용 → 브랜드 전체 비교 불가
- 데이터분석을 진행하기에는 너무나 적은 raw data 개수
- 긍정/부정 데이터 간의 큰 unbalance
- 텍스트 분석 한계: 역접 접속사·부정 부사 카운팅 방식 → 문맥·아이러니 미반영

개선 방향

- 다제품 확장: 여러 제품에 방정식 적용 → 제품 간 리스크 순위 비교
- 다브랜드 비교: 경쟁 브랜드 크롤링 → 시장 내 포지셔닝 파악
- LLM 기반 텍스트 분석: 단순 카운팅 → GPT 기반 감성 분석으로 정교화
- 실시간 대시보드: 새 리뷰가 올라올 때마다 자동으로 Y 점수 업데이트

마무리 - 프로젝트 성과

	기존 방식	프로젝트
판단 기준	별점·직감	데이터 기반 복합 리스크 점수
문제 파악	어떤 제품이 안 좋다	왜 안 좋은지 + 어떤 유형인지까지
우선 순위	경험에 의존	마스터 방정식으로 자동 도출
재현성	사람마다 다름	언제 돌려도 동일한 결과
액션	포괄적 대응	클러스터별 맞춤 처방

“별점이 숨기고 있는 진짜 불만을 찾아내고,
브랜드가 한정된 자원을 가장 효율적으로 쓸 수 있도록 안내하는 R&D 우선순위 엔진”

Thank you